



Rapport de Projet de stage

Titre du projet :

Développement d'une application web de réservation
vétérinaire



Réalisé par :

Jabloun Oumaima

Année Universitaire 2025 – 2026

Table des matières

1	Introduction générale	1
1.1	Cadre général du projet	2
1.2	Présentation de l'organisme d'accueil	2
1.2.1	Présentation Clinique Vétérinaire Kélibia	2
1.2.2	Organigramme de la clinique vétérinaire	2
1.2.3	Présentation du groupe Vétérinaire Kélibia et Pharmavie Kélibia	2
1.2.4	Activités et produits	3
1.2.5	Les différentes directions	3
1.3	Etude de l'existant	3
1.4	Cadre générale du projet	3
1.4.1	Problématique	3
1.4.2	Solution	4
1.5	Synthèse pour le choix de la solution	4
1.6	Scrum	7
1.6.1	Principe du Scrum	7
1.6.2	Les rôles Scrum	7
1.6.3	Backlog de produit :	8
1.7	Conclusion	8
2	Analyse et spécification des besoins	9
2.1	Spécification des besoins	9
2.1.1	Identification des acteurs	9
2.1.2	Besoins fonctionnels des acteurs	9
2.1.3	Besoins non fonctionnels	10
2.2	Diagramme de cas d'utilisation global	11
2.3	Diagramme de Classe	12
2.4	Planification " Scrum"	13
2.4.1	Planification des sprints	14
2.4.2	Le Planning	14
2.5	Backlog Produit	15
2.6	Conclusion	16
3	Architecture logicielle et Framework de développement	17
3.1	Choix technologiques	17
3.1.1	Outils de modélisation	17
3.1.2	Outils de developpement	18

3.1.3	Framework et langages de développement	19
3.2	Architecture de l'application	22
3.2.1	Architecture MVT	22
3.3	Conclusion	22
4	Authentification	23
4.1	sprint 2 : Authentification	23
4.1.1	Backlog du sprint 2	23
4.1.2	Spécification et conception du sprint	24
4.2	sprint 3 : Gestion des animaux	29
4.2.1	Spécification et conception du sprint	30
4.3	sprint 4 : Prise de rendez-vous	34
4.3.1	Backlog Produit	34
4.3.2	Spécification et conception du sprint	35
4.4	Conclusion	36
5	Présentation des interfaces et fonctionnalités de l'application	37
5.1	Page d'accueil	37
5.2	Page d'authentification (connexion / inscription)	38
5.3	Processus de Création de Nouveau Rendez-vous	41
5.4	Interface Administrateur	46
5.5	Conclusion	59
5.6	Conclusion Générale	59

Chapitre 1

Introduction générale

Dans un contexte de transformation numérique croissante, les secteurs traditionnels, y compris la santé animale, sont appelés à moderniser leurs processus pour répondre aux attentes des usagers. Ce projet s'inscrit dans cette dynamique en proposant une solution digitale innovante pour la gestion des rendez-vous vétérinaires.

L'objectif est de concevoir et de développer un système de réservation en ligne permettant aux propriétaires d'animaux de prendre, modifier et suivre leurs rendez-vous de manière simple, rapide et sécurisée, tout en offrant au personnel de la clinique un outil administratif efficace pour gérer les demandes.

Ce document présente le cadre général du projet, l'étude de l'existant, la problématique identifiée, la solution proposée, ainsi que la méthodologie de développement adoptée (Scrum). Il se conclut par une synthèse justifiant le choix de la solution technique retenue.

1.1 Cadre général du projet

La clinique vétérinaire est un établissement spécialisé dans la santé et le bien-être des animaux de compagnie (chiens, chats, petits mammifères, etc.). Elle propose une gamme complète de services : consultations, vaccinations, chirurgies, soins dentaires, hospitalisation et conseils nutritionnels.

Avec une clientèle fidèle et en croissance, la clinique fait face à une charge administrative importante, notamment dans la gestion des appels téléphoniques pour les rendez-vous.

1.2 Présentation de l'organisme d'accueil

Nous présentons dans cette partie la clinique vétérinaire, son organisation, ses activités principales ainsi que ses différentes directions.

1.2.1 Présentation Clinique Vétérinaire Kélibia

La clinique vétérinaire est un établissement spécialisé dans la santé et le bien-être des animaux de compagnie (chiens, chats, petits mammifères, etc.). Elle propose une gamme complète de services : consultations, vaccinations, chirurgies, soins dentaires, hospitalisation et conseils nutritionnels.

Avec une clientèle fidèle et en croissance, la clinique fait face à une charge administrative importante, notamment dans la gestion des appels téléphoniques pour les rendez-vous.

1.2.2 Organigramme de la clinique vétérinaire

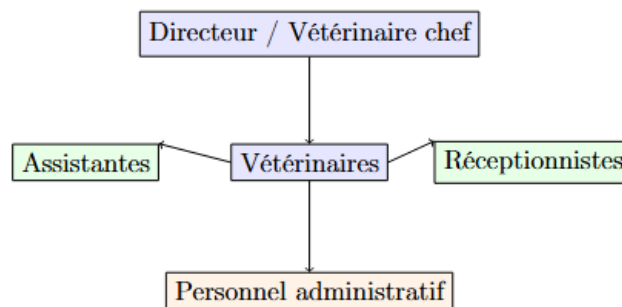


FIGURE 1.1 – Organigramme de la clinique vétérinaire

1.2.3 Présentation du groupe Vétérinaire Kélibia et Pharmavie Kélibia

Le projet s'inspire du fonctionnement de la Clinique Vétérinaire Kélibia, située à Ezzahra (Kélibia), et de son partenaire Pharmavie, située à Kélibia. Ces deux entités collaborent étroitement pour offrir un service complet, allant du diagnostic au traitement,

en passant par la vente de produits dédiés aux animaux de compagnie : alimentation (croquettes pour chiens et chats), cosmétiques, ainsi qu’accessoires (colliers, laisse, etc.).

L’intégration d’un système de gestion numérique renforcerait leur synergie et améliorerait l’expérience client.

1.2.4 Activités et produits

Activités principales :

- Consultations générales et spécialisées
- Vaccinations et prévention
- Chirurgie et hospitalisation
- Stérilisation
- Conseils comportementaux et nutritionnels

Produits vendus (via Pharmavie) :

- Aliments spécifiques (croquettes, compléments)
- Accessoires (colliers, jouets, harnais)
- Produits cosmétiques et de beauté.

1.2.5 Les différentes directions

La clinique est divisée en plusieurs pôles fonctionnels :

- **Direction médicale** : supervision des soins
- **Direction administrative** : gestion des RDV, facturation
- **Direction communication** : relation client, campagnes

1.3 Etude de l’existant

Actuellement, la prise de rendez-vous se fait majoritairement **par téléphone**. Ce système présente plusieurs limites :

- Surcharge des lignes téléphoniques
- Erreurs de transmission (date, heure, animal)
- Temps perdu pour les réceptionnistes
- Difficulté de suivi des historiques
- Absence de planning visuel en temps réel

De plus, il n’existe pas d’outil centralisé permettant aux clients de consulter leurs rendez-vous passés ou de reprogrammer facilement un RDV annulé.

1.4 Cadre générale du projet

1.4.1 Problématique

Problématique

Comment **digitaliser la gestion des rendez-vous vétérinaires** afin de :

- Réduire la charge administrative liée aux appels ?
- Améliorer l’expérience utilisateur ?

- Centraliser l'historique médical des animaux ?
- Offrir un outil de validation et de suivi pour le personnel ?

Sans solution numérique, la clinique risque de perdre en efficacité et en satisfaction client, surtout face à la montée des plateformes de santé animale en ligne.

1.4.2 Solution

Nous proposons le développement d'une **application web de réservation vétérinaire** basée sur le framework **Django (Python)**, comprenant deux interfaces :

- Une **interface utilisateur** pour les propriétaires d'animaux
- Une **interface d'administration** pour le personnel de la clinique

Les fonctionnalités clés incluent :

- Prise de rendez-vous en ligne
- Gestion des animaux par propriétaire
- Validation ou refus des RDV par l'admin
- Téléchargement de l'historique vétérinaire en PDF

1.5 Synthèse pour le choix de la solution

Face à la problématique de gestion manuelle et inefficace des rendez-vous vétérinaires, plusieurs solutions ont été envisagées :

Un simple formulaire Google Forms

Une application mobile

Une application web autonome

Après analyse des besoins fonctionnels, techniques et budgétaires, la solution retenue est un **système web de réservation développé avec Django (Python)**.

Critères d'évaluation et justification du choix

Critère	Solutions envisagées	Solution retenue : Application web avec Django
Facilité d'accès	Google Forms : accessible mais limité / App mobile : nécessite téléchargement	Accès depuis tout appareil (PC, tablette, smartphone) sans installation
Sécurité	Google Forms : faible contrôle / App mobile : dépend de la plateforme	Authentification sécurisée, gestion des permissions, protection CSRF
Évolutivité	Google Forms : très limité / App mobile : coûteuse à maintenir	Architecture modulaire (MVT), facile à étendre (PDF, rappels, paiement, etc.)
Personnalisation	Google Forms : interface basique / App mobile : design figé	Design personnalisé (CSS, Bootstrap), interface admin sur mesure
Coût de développement	App mobile : élevé (iOS + Android)	Faible coût : outils gratuits (Django, SQLite, Render.com)
Maintenance	Google Forms : aucune logique métier	Facile à mettre à jour grâce à la structure claire de Django
Stockage des données	Google Forms : export difficile	Base de données intégrée (SQLite/PostgreSQL), accès direct aux historiques
Fonctionnalités avancées	Google Forms : non possible	Génération automatique de PDF, validation par l'admin, notifications

TABLE 1.1 – Comparaison des solutions

Pourquoi Django ?

Le framework **Django** a été choisi pour les raisons suivantes :

51 **Rapidité de développement** : il inclut nativement l'authentification, l'administration, les formulaires et les URLs.

51 **Interface d'administration puissante** : `/admin/` permet une gestion intuitive des données.

51 **Sécurité intégrée** : protection contre les injections, CSRF, XSS.

51 **ORM** : permet de travailler avec la base de données sans écrire de SQL.

51 **Communauté active** : documentation riche, nombreuses bibliothèques (ex : **weasyprint** pour le PDF).

51 **Compatible déploiement gratuit** : fonctionne parfaitement sur Render.com ou Railway.app.

Architecture technique retenue MVT

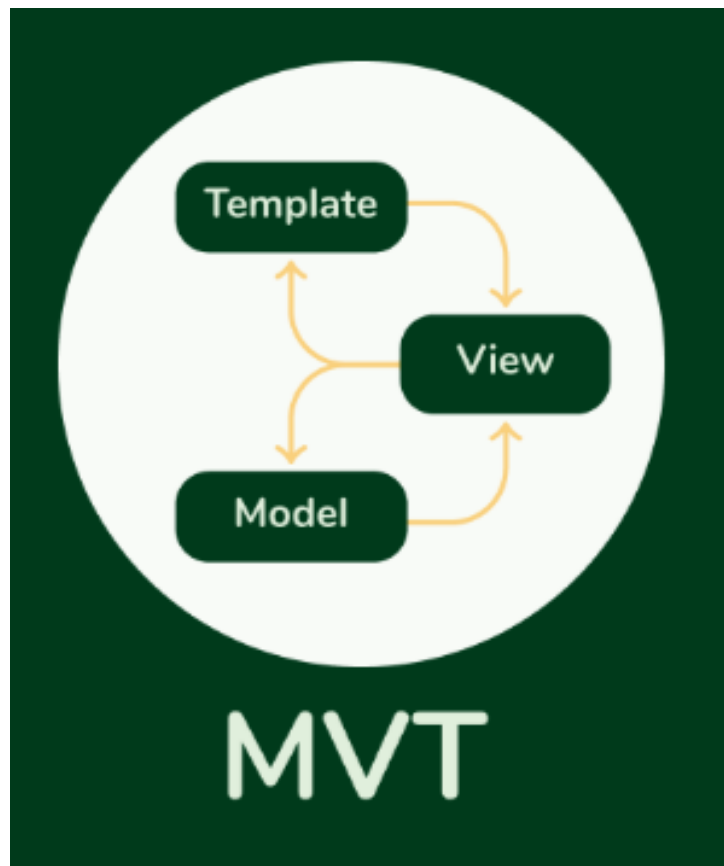


FIGURE 1.2 – Architecture technique retenue MVT

Avantages de la solution choisie

Automatisation : réduction du travail administratif

Accessibilité 24h/24 : les utilisateurs prennent un RDV quand ils veulent

Traçabilité : historique médical par animal en PDF

Contrôle administratif : validation des RDV par le personnel

Évolutif : possibilité d'ajouter des fonctionnalités (rappels, messagerie, paiement)

Limites identifiées et leviers futurs

Limite	Solution future
Utilisation de SQLite en dev	Passage à PostgreSQL en production
Pas de notifications automatiques	Intégration d'emails ou SMS (SendGrid, Twilio)
Pas de planning visuel	Ajout d'un calendrier interactif (FullCalendar.js)

TABLE 1.2 – Limites et améliorations futures

1.6 Scrum

La méthodologie SCRUM est utilisée par de nombreuses entreprises. De même, elle peut être utilisée pour développer des applications Web et mobile. Elle permet entre autres de développer des produits complexes et de produire la plus grande valeur métier dans la durée la plus courte. Elle permet aussi à ses utilisateurs d'être soudés pour accroître la productivité. Pour conclure, cette méthodologie est la plus efficace qui nous permettra de réaliser notre projet dans un temps bien précis.

1.6.1 Principe du Scrum

Scrum en quelques mots :

- Scrum est un processus agile qui permet de produire la plus grande valeur métier dans la durée la plus courte.
- Du logiciel qui fonctionne est produit à chaque sprint (toutes les 2 à 4 semaines).
- Le métier définit les priorités, l'équipe s'organise elle-même pour déterminer la meilleure façon de produire les exigences les plus prioritaires.
- A chaque fin de sprint, tout le monde peut voir fonctionner le produit courant et décider soit de le livrer dans l'état, soit de continuer à l'améliorer pendant un sprint supplémentaire. [2]

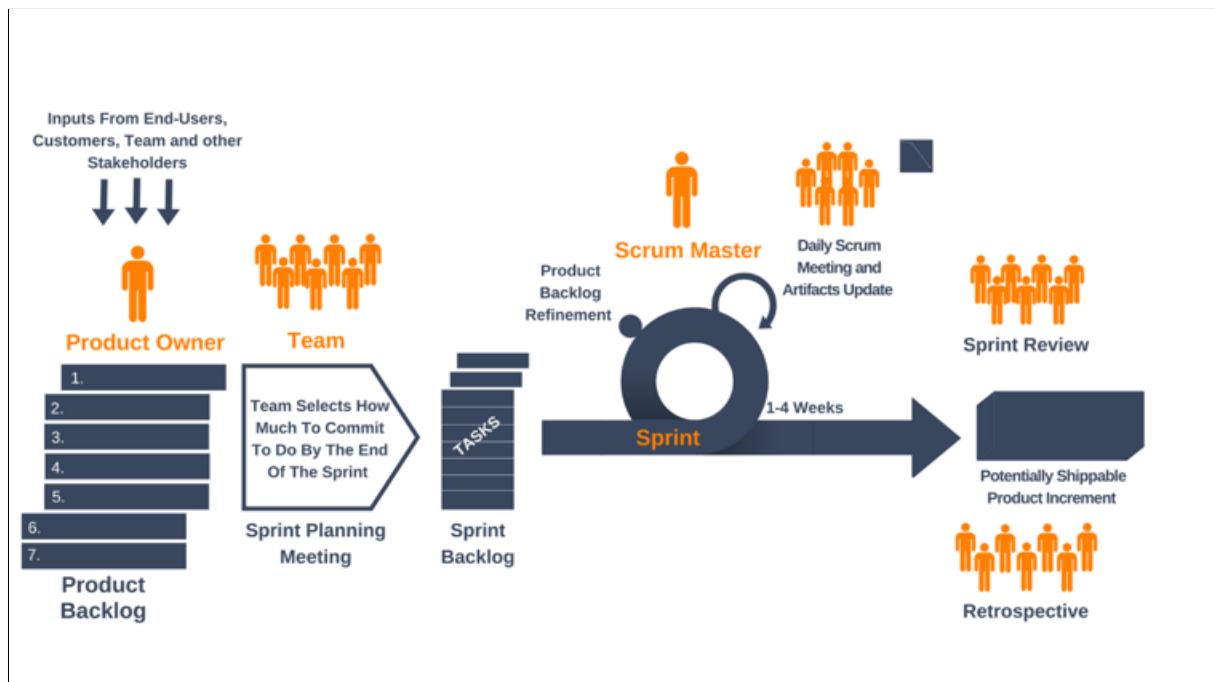


FIGURE 1.3 – Cycle de vie de la méthodologie SCRUM

1.6.2 Les rôles Scrum

D'après la méthode Scrum, chaque collaborateur a un rôle clairement défini dans la gestion du projet, elle définit seulement 3 rôles :

- **Product Owner :**

- Définit les fonctionnalités du produit.
- Choisit la date et le contenu de la release.
- Responsable du retour sur investissement.
- Définit les priorités dans le backlog en fonction de la valeur « métier ».
- Ajuste les fonctionnalités et les priorités à chaque sprint si nécessaire.
- Accepte ou rejette les résultats.
- **Scrum Master :**
 - Représente le management du projet.
 - Responsable de faire appliquer par l'équipe les valeurs et les pratiques de Scrum.
 - Élimine les obstacles.
 - S'assure que l'équipe est complètement fonctionnelle et productive.
 - Facilite une coopération poussée entre tous les rôles et fonctions.
 - Protège l'équipe des interférences extérieures.
- **Equipe de Développement :**
 - Chargée de transformer les besoins exprimés par le Product Owner en fonctionnalités utilisables. Elle est pluridisciplinaire et peut donc encapsuler d'autres rôles tels que développeur, architecte logiciel, DBA, analyste fonctionnel, graphiste/ergonome, ingénieur système.
 - L'équipe s'organise par elle-même.
 - La composition de l'équipe ne doit pas changer pendant un Sprint [3].

1.6.3 Backlog de produit :

Le backlog est élaboré avant le lancement des sprints, dans la phase de préparation (ou sprint 0). Il est utilisé pour planifier la release, puis à chaque sprint, lors de la réunion de planification du sprint pour décider du sous-ensemble qui sera réalisé. C'est donc un outil essentiel pour la planification. Mais il est aussi, par sa nature, un maillon de la gestion des exigences, puisqu'on y collecte ce que doit faire le produit. [4]

1.7 Conclusion

Dans ce premier chapitre, nous avons présenté en premier lieu l'organisme d'accueil et l'étude de l'existant. Ensuite nous avons posé la problématique puis la solution proposée. Enfin nous avons expliqué la méthodologie adoptée. Dans le deuxième chapitre, nous allons entamer le chapitre de spécification des besoins dans lequel nous présenterons les besoins fonctionnels et non fonctionnels, la planification d'équipe scrum, le backlog du produit et les différents cas d'utilisation.

Chapitre 2

Analyse et spécification des besoins

Introduction

Dans ce chapitre, nous allons présenter les besoins fonctionnels et non fonctionnels, le backlog du produit et l'équipe Scrum. Ensuite, nous identifierons le diagramme de classe.

2.1 Spécification des besoins

Dans cette partie, nous présenterons les objectifs de notre application, ce qui nous amène à identifier les acteurs du système et les besoins fonctionnels.

2.1.1 Identification des acteurs

- **Administrateur de l'application** : L'administrateur représente le personnel de la clinique vétérinaire chargé de la gestion opérationnelle des rendez-vous. Il peut s'agir d'un vétérinaire, d'une assistante vétérinaire ou d'un responsable administratif ayant un accès privilégié au système.
- **Utilisateur de l'application** : L'utilisateur est un propriétaire d'animal de compagnie souhaitant prendre, modifier ou suivre ses rendez-vous vétérinaires via une plateforme numérique. Il interagit avec le système via une interface web intuitive et accessible.

2.1.2 Besoins fonctionnels des acteurs

Dans cette partie, nous exposons l'ensemble des besoins fonctionnels des acteurs.

- **Administrateur** :
 - Gérer la validation des rendez-vous.
 - Gérer les fiches animales et leurs rendez-vous associés.
 - Gérer la recherche et le filtrage des données.
 - Gérer les communications internes via des notes.
 - Gérer la génération de documents médicaux.
 - Gérer la traçabilité des actions.

- **Utilisateur :**
 - Gérer l'authentification sécurisée.
 - Gérer son inscription et son profil.
 - Gérer ses animaux (CRUD).
 - Gérer la prise de rendez-vous.
 - Gérer l'historique de ses rendez-vous.
 - Gérer le téléchargement de l'historique PDF.
 - Gérer sa session utilisateur

2.1.3 Besoins non fonctionnels

Les **besoins non fonctionnels** ne concernent pas les fonctions du système (comme prendre un rendez-vous), mais décrivent **comment le système doit bien fonctionner** : qu'il soit sûr, rapide, facile à utiliser, etc.

Voici les principaux critères que le projet respecte :

1. Sécurité

Le mot de passe des utilisateurs est crypté.

Seul l'administrateur peut valider les rendez-vous.

Le site empêche les attaques courantes (ex : injections).

2. Performance

Les pages s'affichent en moins de 3 secondes.

Le site peut gérer plusieurs utilisateurs en même temps.

Il reste fluide même avec beaucoup de données.

3. Ergonomie

Interface simple et intuitive.

Accès depuis téléphone, tablette ou ordinateur (responsive).

Messages d'erreur clairs (ex : « Ce créneau est déjà pris »).

4. Fiabilité

Aucune donnée n'est perdue (rendez-vous, animaux...).

Gestion correcte des erreurs.

Vérification automatique des créneaux doubles.

5. Maintenabilité

Code bien organisé et commenté.

Utilisation de Git pour suivre les modifications.

Facile à corriger ou à améliorer.

6. Évolutivité

Possibilité d'ajouter :

- Rappels par SMS ou email
- Paiement en ligne
- Calendrier interactif
- Connexion avec Google
- Passage facile à PostgreSQL en production.

7. Compatibilité

- Fonctionne sur Chrome, Firefox, Edge, Safari.
- Compatible Windows, Linux, macOS.
- Supporte Python

2.2 Diagramme de cas d'utilisation global

La figure 2.1 représente le diagramme de cas d'utilisation global.

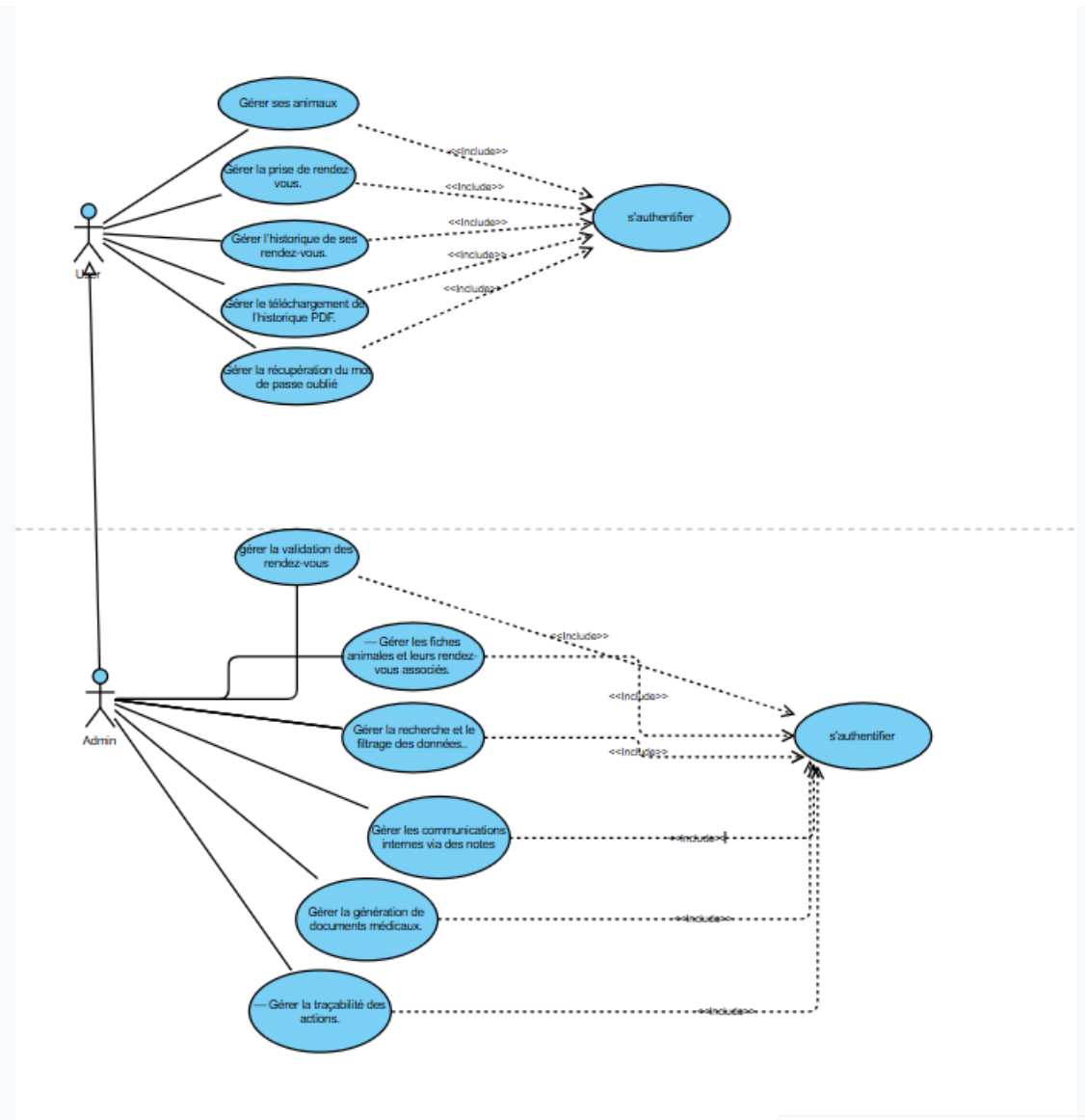


FIGURE 2.1 – Diagramme de Cas D'utilisation Global

2.3 Diagramme de Classe

Le diagramme de classes décrit la perspective structurelle d'un système en illustrant les objets au sein du système, les relations entre ces objets et les attributs et opérations associés à chaque classe d'objets. La figure 2.2 illustre le diagramme de classe global :

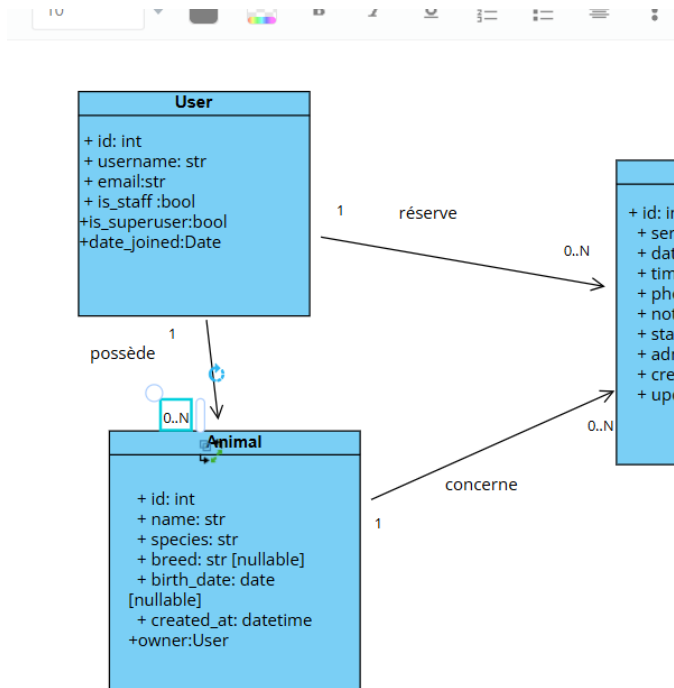


FIGURE 2.2 – Diagramme de Classe Global

Un utilisateur (**User**) peut avoir plusieurs animaux (**Pet**).

→ Chaque animal appartient à un seul propriétaire.

Un utilisateur (**User**) peut créer plusieurs rendez-vous (**Appointment**).

→ Il est responsable de ses demandes de RDV.

Un animal (**Pet**) peut être concerné par plusieurs rendez-vous au fil du temps.

→ Un seul animal par rendez-vous, mais plusieurs visites possibles.

2.4 Planification " Scrum "

Dans cette section, nous allons aborder le pilotage du projet par la méthodologie Scrum. Comme nous l'avons déjà mentionné, nous avons choisi Scrum en raison de sa capacité à permettre un développement rapide et la réutilisabilité des fonctionnalités indépendamment de la plateforme. Cette méthode favorise la collaboration entre différents intervenants. Même en mode individuel, les rôles clés de Scrum ont été simulés mentalement pour assurer une organisation rigoureuse :

- Product Owner : Jabloun Oumaima
- Scrum Master : Jabloun Oumaima
- Équipe de développement : seul Développeur

2.4.1 Planification des sprints

Pour bien organiser notre travail, nous avons besoin de diviser les tâches à effectuer en des sprints. Les sprints sont des périodes d'un mois au maximum, au bout de laquelle nous délivrons un incrément du produit, potentiellement livrable. La figure 2.3 représente la répartition des sprints :

Sprint N°	Période	Tâches / Objectifs
Sprint 1	Semaine 1	Analyse & Conception
Sprint 2	Semaine 2-3	Authentification & Gestion des utilisateurs
Sprint 3	Semaine 4	Gestion des animaux
Sprint 4	Semaine 5	Formulaire de prise de rendez-vous
Sprint 5	Semaine 6-7	Interface d'administration & Tests finaux

FIGURE 2.3 – Planification des Sprints

2.4.2 Le Planning

Dans la figure 2.4 présente notre planification de chaque sprint.

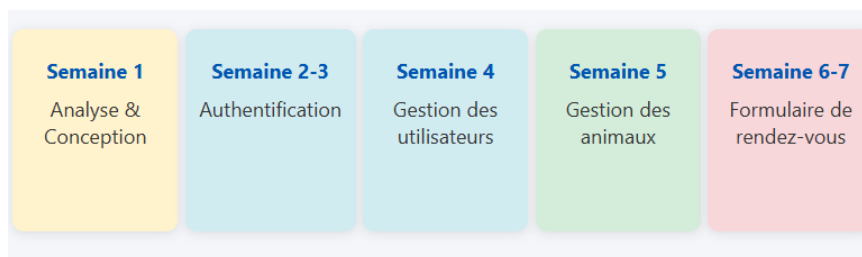


FIGURE 2.4 – Planning du déroulement du projet

[a4paper,11pt]article [utf8]inputenc [T1]fontenc [french]babel array tabularx xcolor
longtable geometry margin=1in

2.5 Backlog Produit

Le tableau 2.1 présente le backlog du produit de notre projet, qui détaille les différentes user stories. Il est important de souligner que le backlog du produit est un outil essentiel pour la planification et la gestion des exigences, car il regroupe toutes les fonctionnalités et les tâches à réaliser pour le produit.

article

longtable multirow caption array ragged2e [table]xcolor

TABLE 2.1 – Backlog Général du Produit

Id	Sprint	Id	Fonctionnalités	Priorité
1	Sprint N°1 :	1.1	En tant que développeur, je veux faire la création de projet Django.	Moyenne
	Analyse	1.2	En tant que développeur, je veux faire l'analyse des besoins.	Moyenne
2	Sprint N°2 :	2.1	En tant que visiteur, je veux pouvoir m'inscrire pour créer un compte utilisateur.	Élevée
	Authentification	2.2	En tant qu'utilisateur, je veux me connecter à mon compte de manière sécurisée.	Élevée
		2.3	En tant qu'utilisateur, j'ai oublié mon mot de passe ; je veux pouvoir le réinitialiser par email.	Élevée
3	Sprint N°3 :	3.1	En tant qu'utilisateur, je veux ajouter un animal (nom, espèce, race, date de naissance).	Élevée
	Gestion des animaux	3.2	En tant qu'utilisateur, je veux modifier ou supprimer un animal de ma liste.	Élevée
4	Sprint N°4 :	4.1	En tant qu'utilisateur, je veux prendre un rendez-vous pour l'un de mes animaux.	Élevée
	Formulaire de prise de rendez-vous	4.2	En tant qu'utilisateur, je veux voir l'historique de mes rendez-vous et attendre la confirmation ou le refus.	Élevée
		4.3	En tant qu'utilisateur, je veux télécharger l'historique médical de mon animal en PDF.	Élevée
5	Sprint N°5 : Interface d'administration	5.1	En tant qu'administrateur, je veux valider un rendez-vous soumis par un utilisateur.	Élevée
		5.2	En tant qu'administrateur, je veux refuser un rendez-vous avec une note explicative.	Élevée
		5.3	En tant qu'administrateur, je veux consulter toutes les fiches animales.	Élevée
		5.4	En tant qu'administrateur, je veux filtrer les rendez-vous par statut, date ou utilisateur.	Élevée
		5.5	En tant qu'administrateur, je veux accéder à une interface personnalisée (logo, thème).	Élevée

2.6 Conclusion

Dans ce chapitre, nous avons présenté tout d'abord les besoins fonctionnels et non fonctionnels, le diagramme de classe, la planification "scrum", et le backlog du produit. Le prochain chapitre va présenter les technologies utilisées et la réalisation de premier sprint.

Chapitre 3

Architecture logicielle et Framework de développement

Introduction

Dans ce chapitre, nous allons présenter les choix des technologies, l'architecture de notre application, ainsi que son contexte technique lors du sprint 0.

3.1 Choix technologiques

Dans cette étape, nous allons entamer la partie implémentation en présentant outil de modélisation, les outils de développements et les langages de programmation utilisés pour la réalisation du projet.

3.1.1 Outils de modélisation

StarUML

StarUML est un logiciel de modélisation UML (Unified Modeling Language) open source qui peut remplacer dans bien des situations des logiciels commerciaux et coûteux comme Rational Rose¹ ou Together². Étant simple d'utilisation, nécessitant peu de ressources système, supportant UML 2, ce logiciel constitue une excellente option pour une familiarisation à la modélisation.



FIGURE 3.1 – Logo StarUML

plantuml

PlantUML est un outil qui permet de créer facilement des diagrammes UML et autres schémas à partir d'un simple texte descriptif

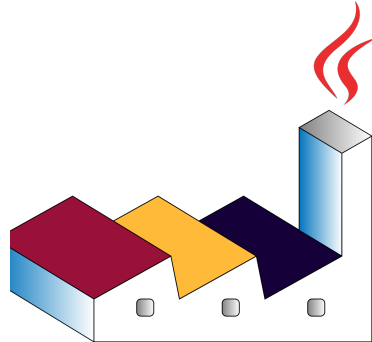


FIGURE 3.2 – Logo PlantUML

3.1.2 Outils de developpement

Dans cette partie, nous allons présenter les différents logiciels utilisés pour la réalisation des applications.

Cursor

Cursor est un éditeur de code intelligent propulsé par l'IA, conçu pour révolutionner la façon dont les développeurs écrivent, déboguent et optimisent leur code. Combinant les fonctionnalités avancées d'un IDE (Environnement de Développement Intégré) avec la puissance de l'intelligence artificielle, Cursor se positionne comme un outil incontournable pour les programmeurs modernes.



FIGURE 3.3 – Logo Cursor

3.1.3 Framework et langages de développement

Python

Python est un langage de programmation interprété, de haut niveau, polyvalent et orienté objet. Il est connu pour sa syntaxe claire et facile à lire, ce qui le rend efficace pour le développement rapide, l'apprentissage et l'automatisation



FIGURE 3.4 – Logo Python

sqlite

SQLite3 est une base de données légère et sans serveur qui stocke toutes les données dans un seul fichier et utilise le langage SQL pour gérer ces données. (Intégré nativement dans Django)



FIGURE 3.5 – Logo StatrUml

sqlite

Utiliser le système de gestion de base de données **PostgreSQL** à l'intérieur d'un conteneur Docker. (C'est une manière **rapide et pratique** d'installer, d'exécuter et de gérer

PostgreSQL sans avoir besoin de l'installer directement sur votre ordinateur.)
Docker crée un environnement isolé (appelé **conteneur**) qui contient tout le nécessaire pour que PostgreSQL fonctionne : le serveur, les fichiers de configuration et les données.

PostgreSQL with Docker



FIGURE 3.6 – Logo postgress avec docker

Django

un framework web Python gratuit et open source qui permet aux développeurs de créer rapidement des sites web et des applications web sécurisés et évolutifs.



FIGURE 3.7 – Logo StatrUml

HTML

Le HTML (HyperText Markup Language) est un langage de balisage standard utilisé pour structurer et présenter le contenu des pages web. Il ne s'agit pas d'un langage de programmation, mais d'un langage de description qui utilise des balises pour organiser les informations comme les titres, les paragraphes, les images, les tableaux et les liens, afin qu'elles soient affichées correctement par un navigateur web.



FIGURE 3.8 – Logo html

CSS

Le CSS (Cascading Style Sheets) est un langage utilisé pour décrire l'apparence et la mise en page des pages web. Il permet de contrôler les couleurs, les polices, les marges, les tailles ainsi que la disposition des éléments afin de rendre un site plus attractif et facile à utiliser. Grâce au CSS, le contenu (HTML) est séparé de la présentation, ce qui rend la conception de sites plus claire, plus organisée et adaptable aux différents écrans (ordinateurs, tablettes, smartphones).



FIGURE 3.9 – Logo css

Bootstrap

Le Bootstrap est un ensemble des outils utilisé pour la création du graphisme de site ou d'application web.



FIGURE 3.10 – Logo Bootstrap

3.2 Architecture de l'application

3.2.1 Architecture MVT

Django utilise un modèle appelé MVT (Modèle-Vue-Template), équivalent au modèle MVC utilisé dans d'autres frameworks.

Le projet utilise l'architecture MVT, utilisée par Django. Elle divise l'application en trois parties :

- Modèle : gère les données (comme les animaux et les rendez-vous).
- Vue : contient la logique (par exemple, enregistrer un RDV ou vérifier la connexion).
- Temple : c'est ce que l'utilisateur voit (les pages web en HTML).

Cette séparation permet de mieux organiser le code et de rendre le site plus facile à modifier et à améliorer

3.3 Conclusion

Dans ce chapitre, nous avons présenté les outils et technologies choisis pour développer notre application. Nous utilisons StarUML et PlantUML pour la modélisation, l'éditeur Cursor pour coder plus efficacement, et le langage Python avec le framework Django pour construire l'application.

L'architecture MVT de Django nous permet de bien séparer les données, la logique et l'interface utilisateur. Côté base de données, nous commençons avec SQLite et prévoyons d'utiliser PostgreSQL dans un conteneur Docker pour la production. Enfin, l'interface web est réalisée avec HTML, CSS et Bootstrap pour être claire et adaptée à tous les écrans. Ces choix forment une base solide, simple et évolutive pour la suite du projet.

Chapitre 4

Authentification

Introduction

Au cours de ce chapitre, nous allons présenter les différentes étapes de réalisation du premier sprint "Authentification" pour admin et utilisateur

4.1 sprint 2 : Authentification

1. Objectifs du sprint 2

Le Sprint 2 est consacré à la mise en place du système d'authentification des utilisateurs. L'objectif principal est de permettre aux visiteurs de créer un compte, de se connecter et de gérer leur accès de manière sécurisée. Durant ce sprint, les fonctionnalités suivantes sont développées

- Le formulaire d'inscription avec validation des données,
- La connexion et la déconnexion sécurisées via le système d'authentification intégré à Django .
- La gestion du mot de passe oublié (réinitialisation par email).

Des mesures de sécurité sont également mises en œuvre, telles que la protection CSRF et le hachage des mots de passe. Ce sprint garantit que chaque utilisateur peut accéder à son espace personnel, ouvrant ainsi la voie à la gestion des animaux et des rendez-vous dans les sprints suivants.

4.1.1 Backlog du sprint 2

Le tableau 3.1 contient une liste des tâches identifiées par nous qui devront être réalisées avant la fin de sprint.

Tâches	Priorité
- Créer le formulaire d'inscription (CustomUserCreationForm)	Moyenne
- Mettre en place la connexion sécurisée avec Django auth	Elevée

Tâches	Priorité
Implémenter la déconnexion (logout)	Moyenne
Gérer la réinitialisation du mot de passe (mot de passe oublié)	Elevée
Configurer l'envoi d'email via le backend (console ou SMTP)	Elevée
Personnaliser les templates	Moyenne
Ajouter la protection CSRF et valider la sécurité des sessions	Elevée
Tester les cas d'utilisation :Inscription réussie-Connexion échouée-Réinitialisation du mot de passe	Moyenne

TABLE 4.1 – Backlog du Sprint 2

4.1.2 Spécification et conception du sprint

Dans cette section, nous allons présenter la spécification et l'analyse du sprint, en mettant en évidence les différents aspects du développement. Nous aborderons notamment le diagramme de cas d'utilisation, ainsi que les modèles statique et dynamique associés à ce sprint.

Diagramme de cas d'utilisation

La figure 3.13 représente le diagramme de cas d'utilisation de ce sprint.

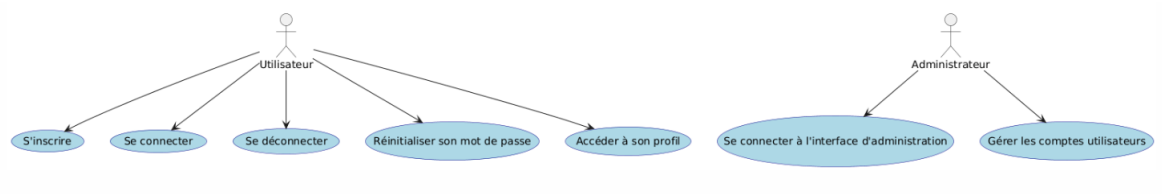


FIGURE 4.1 – Diagramme de cas d'utilisation sprint (2)

Cas d'utilisation	Description
Utilisateur	— S'inscrire : Créer un compte en fournissant un nom d'utilisateur, une adresse e-mail et un mot de passe. — Se connecter : Accéder à son espace personnel en utilisant ses identifiants (email et mot de passe). — Se déconnecter : Fermer sa session pour garantir la sécurité de son compte, notamment sur un appareil partagé. — Réinitialiser son mot de passe : En cas d'oubli, demander un lien de réinitialisation envoyé par email. — Accéder à son profil : Consulter ses informations personnelles (nom, email).
Administrateur	— Se connecter à l'interface d'administration : Accéder à /admin/ après authentification. — Gérer les comptes utilisateurs : Consulter la liste des utilisateurs inscrits (lecture seule dans ce sprint).

TABLE 4.2 – Description Cas d'utilisation Sprint (2)

Diagramme de Séquence

Le diagramme de séquence est utilisé pour représenter les vues dynamiques du système, en se concentrant sur la chronologie des envois de messages entre les objets.

Diagramme de séquence pour le scénario « Authentification »

Le Sprint 2 est consacré à la mise en place du système d'authentification. Le diagramme de séquence illustre les interactions entre l'utilisateur et le système lors des principales actions liées au compte utilisateur :

- L'inscription
- La connexion

- La réinitialisation du mot de passe

Il montre l'ordre chronologique des messages échangés entre les composants du système (navigateur, vues Django, formulaires, base de données, serveur d'email).

1. Scénario 1 : Inscription d'un nouvel utilisateur

Objectif : Permettre à un visiteur de créer un compte.

Étapes :

1. L'utilisateur accède à la page `/signup/` via son navigateur.
2. Le navigateur demande au serveur Django d'afficher la vue *SignupView*.
3. Cette vue initialise le formulaire *CustomUserCreationForm*.
4. L'utilisateur remplit ses informations (nom d'utilisateur, email, mot de passe).
5. Le navigateur soumet les données au formulaire.
6. Le formulaire valide les champs (ex : format email, complexité du mot de passe).
7. Si tout est correct, le nouveau compte est créé dans la base de données (*User*) avec le mot de passe haché (sécurisé).
8. Une fois l'enregistrement terminé, l'utilisateur est redirigé vers la page de connexion.

Résultat : Un nouveau compte est créé, sécurisé et prêt à être utilisé.

2. Scénario 2 : Connexion à un compte existant

Objectif : Permettre à un utilisateur inscrit de se connecter.

Étapes :

1. L'utilisateur accède à `/login/`.
2. Le serveur affiche le formulaire de connexion (*LoginView*).
3. L'utilisateur saisit son email et son mot de passe.
4. Le navigateur envoie ces informations au *LoginForm*.
5. Ce formulaire utilise la fonction `authenticate()` de Django pour vérifier les identifiants dans la base de données.

Si les identifiants sont corrects :

- La session est démarrée
- L'utilisateur est redirigé vers l'accueil

×Sinon :

- Un message d'erreur s'affiche
- L'utilisateur doit recommencer

Résultat : L'utilisateur accède à son espace personnel de manière sécurisée.

3. Scénario 3 : Réinitialisation du mot de passe

Objectif : Aider un utilisateur qui a oublié son mot de passe.

Étapes :

1. L'utilisateur clique sur « Mot de passe oublié ? ».
2. Il est redirigé vers *PasswordResetView*, qui affiche un formulaire.
3. L'utilisateur entre son adresse email.
4. Le système vérifie si cet email existe dans la base de données.
5. Si oui, un lien de réinitialisation sécurisé est généré (avec un token signé et temporaire).
6. Ce lien est envoyé par email via le serveur d'email.
7. L'utilisateur reçoit l'email et clique sur le lien.
8. Il arrive sur *PasswordResetConfirmView*, où il peut saisir un nouveau mot de passe.
9. Le formulaire *SetPasswordForm* met à jour le mot de passe (haché) dans la base de données.
10. L'utilisateur est redirigé vers une page de confirmation.

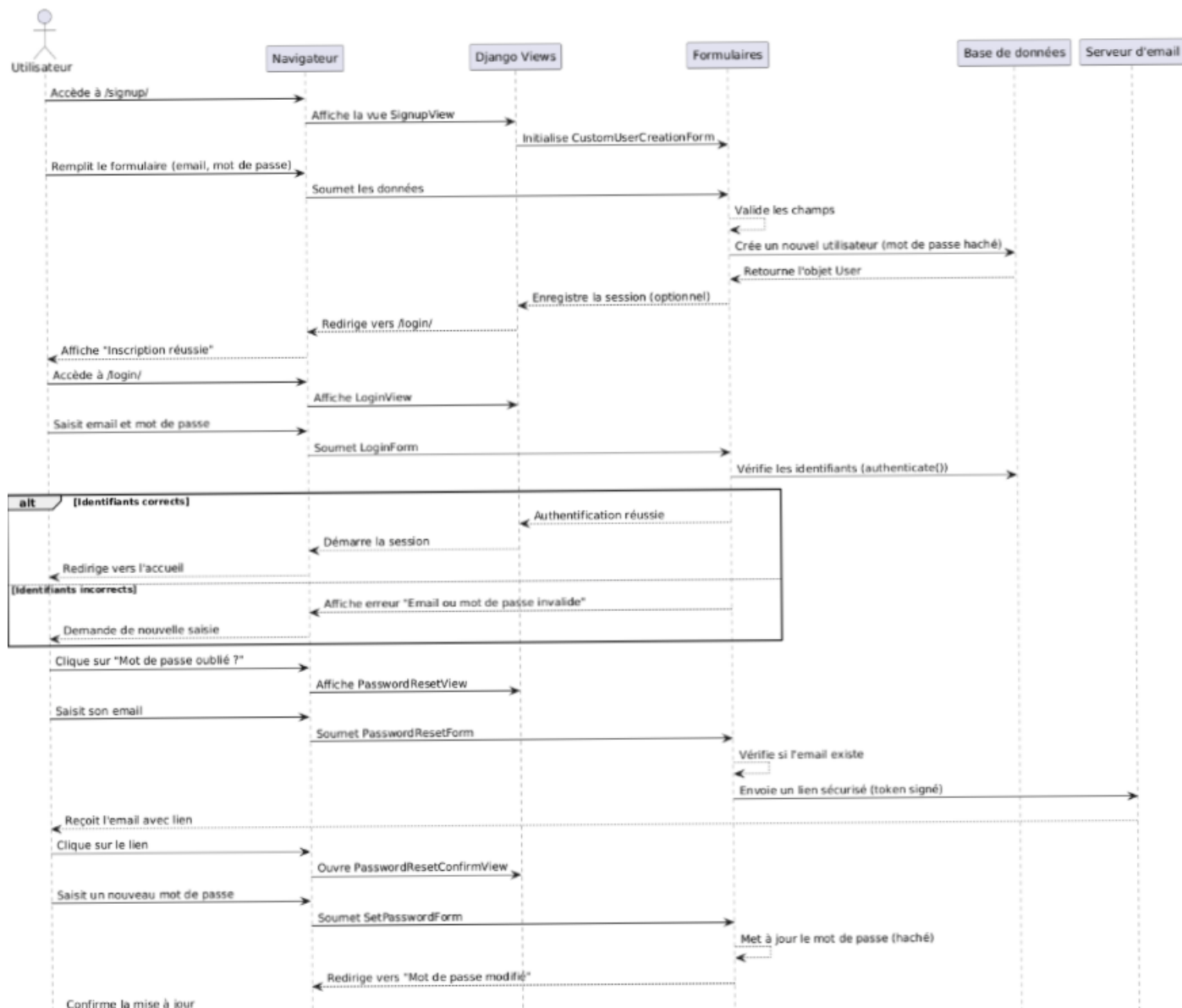


FIGURE 4.2 – Diagramme de Séquence sprint (2)

Diagramme de Classe – Sprint 2 : Authentification

Ce diagramme montre les composants principaux du système d'authentification développé pendant le Sprint 2. Il met en lumière les interactions entre les classes qui permettent aux utilisateurs de :

1. -S'inscrire
2. -Se connecter
3. -Réinitialiser leur mot de passe

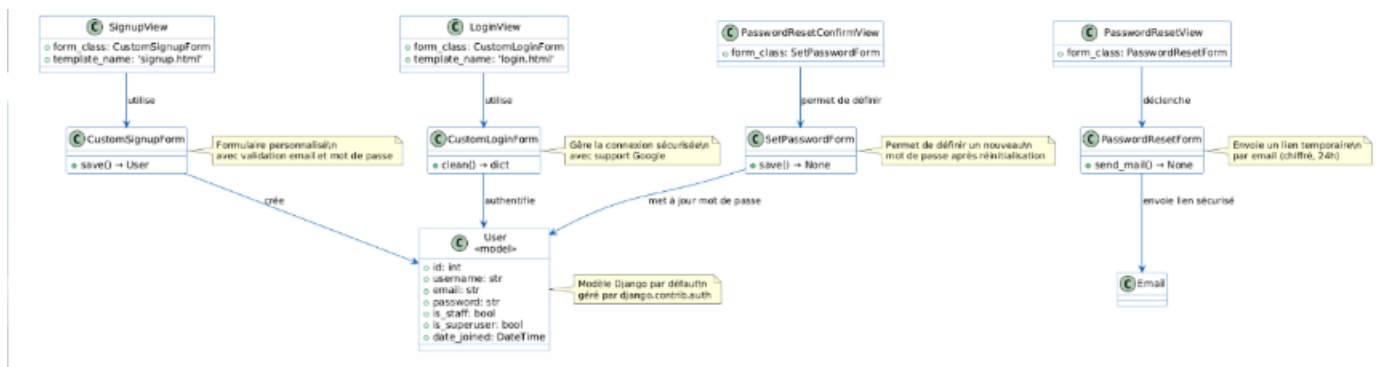


FIGURE 4.3 – Diagramme de Classe sprint (2)

4.2 sprint 3 : Gestion des animaux

1. Objectifs du sprint 3

Permettre à chaque utilisateur de gérer ses animaux domestiques (ajout, modification, suppression) dans son espace personnel.

Ce sprint vise à créer la fiche d'identité de chaque animal, base essentielle pour les rendez-vous médicaux ultérieurs.

1. Objectifs du sprint 3

Le tableau 3.2 contient une liste des tâches identifiées par nous qui devront être réalisées avant la fin de sprint.

Tâches	Priorité
- Créer le modèle Pet dans models.py	Elevée
- Définir les champs : nom, espèce, race, date de naissance, propriétaire	Elevée
- Appliquer les migrations(makemigrations,migrate)	Elevée
- Créer le formulaire PetForm basé sur le modèle	Moyenne
- Créer la vue add _{pet} (POST/GET)	Moyenne
- Créer le template add _{pet.html} avec Bootstrap	Moyenne
- Lier l'animal au propriétaire connecté (request.user)	Elevée

Tâches	Priorité
- Valider que la date de naissance n'est pas future	Moyenne
- Tester l'ajout/modification/suppression d'un animal	faible
- Mettre à jour le menu utilisateur avec un lien vers "Mes animaux"	faible

TABLE 4.3 – Backlog du Sprint 3

4.2.1 Spécification et conception du sprint

Dans cette section, nous allons présenter la spécification et l'analyse du sprint, en mettant en évidence les différents aspects du développement. Nous aborderons notamment le diagramme de cas d'utilisation, ainsi que les modèles statique et dynamique associés à ce sprint.

Diagramme de cas d'utilisation

La figure 3.13 représente le diagramme de cas d'utilisation de ce sprint.

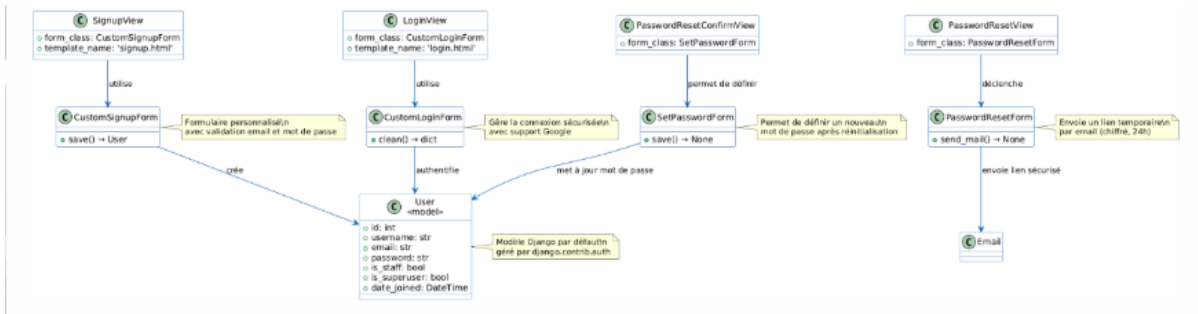


FIGURE 4.4 – Diagramme de cas d'utilisation sprint (3)

ID	Cas d'utilisation	Acteur	Description
Ajouter un animal	— Objectif : Créer une fiche pour un nouvel animal — Pré-conditions : race — Scénario principal : L'utilisateur clique sur "Ajouter un animal", saisit les informations (nom, espèce, date de naissance), puis soumet le formulaire — Post-conditions : Un nouvel animal est ajouté à la liste de l'utilisateur — Exceptions : Champ obligatoire manquant, date invalide, doublon	Utilisateur	Ajouter un animal
Modifier un animal	— Objectif : Mettre à jour les informations d'un animal existant — Pré-conditions : L'animal existe et appartient à l'utilisateur — Scénario principal : L'utilisateur sélectionne un animal puis modifie les informations — Post-conditions : Les informations de l'animal sont mises à jour — Exceptions : Données invalides, tentative de modification non autorisée	Utilisateur	Modifier un animal

ID	Cas d'utilisation	Acteur	Description
Supprimer un animal	— Objectif : Retirer un animal de sa liste — Pré-conditions : L'animal existe et appartient à l'utilisateur — Scénario principal : L'utilisateur sélectionne un animal puis confirme la suppression — Post-conditions : L'animal est supprimé de la base — Exceptions : Annulation par l'utilisateur, erreur de suppression	Utilisateur	Supprimer un animal
Consulter mes animaux	— Objectif : Visualiser la liste de ses animaux — Pré-conditions : Être connecté — Scénario principal : L'utilisateur accède à son tableau de bord et consulte ses animaux — Post-conditions : La liste des animaux est affichée — Exceptions : Aucun animal enregistré → message "Aucun animal trouvé"	Utilisateur	Consulter mes animaux
Consulter tous les animaux	— Objectif : Voir toutes les fiches animales du système — Pré-conditions : Être connecté comme administrateur — Scénario principal : L'administrateur se connecte à /admin/ et consulte les animaux — Post-conditions : Liste complète des animaux triable et filtrable — Exceptions : Accès refusé si non authentifié comme staff	Administrateur	Consulter tous les animaux

TABLE 4.4 – Description des Cas d'utilisation (Gestion des animaux)

Diagramme de Séquence

Le diagramme de séquence est utilisé pour représenter les vues dynamiques du système, en se concentrant sur la chronologie des envois de messages entre les objets.

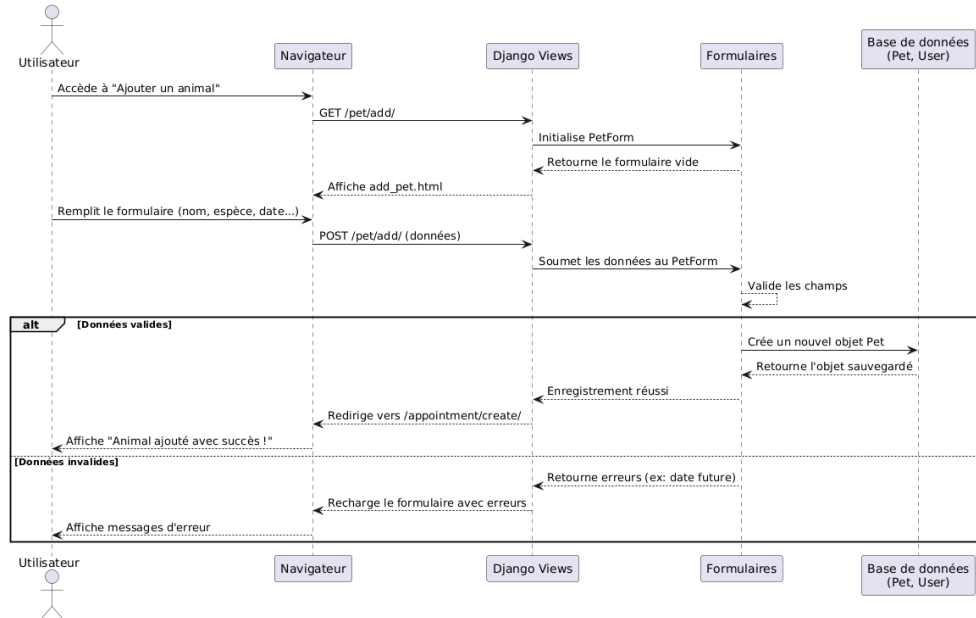


FIGURE 4.5 – Diagramme de Séquenc sprint 3

Étape	Description
1	L'utilisateur demande à ajouter un animal
2	Le serveur affiche un formulaire vide
3	L'utilisateur remplit les informations
4	Le formulaire est soumis au serveur
5	Validation des données (ex : date de naissance)
6	Si OK → Sauvegarde dans la base de données
7	Redirection avec message de succès

TABLE 4.5 – Processus d'ajout d'un animal

Diagramme de Classe

Voici le Diagramme de Classe .Ce sprint est consacré à la création et gestion des fiches animales (Pet) liées aux utilisateurs.

Diagramme de Classe «Gestion des Animaux»

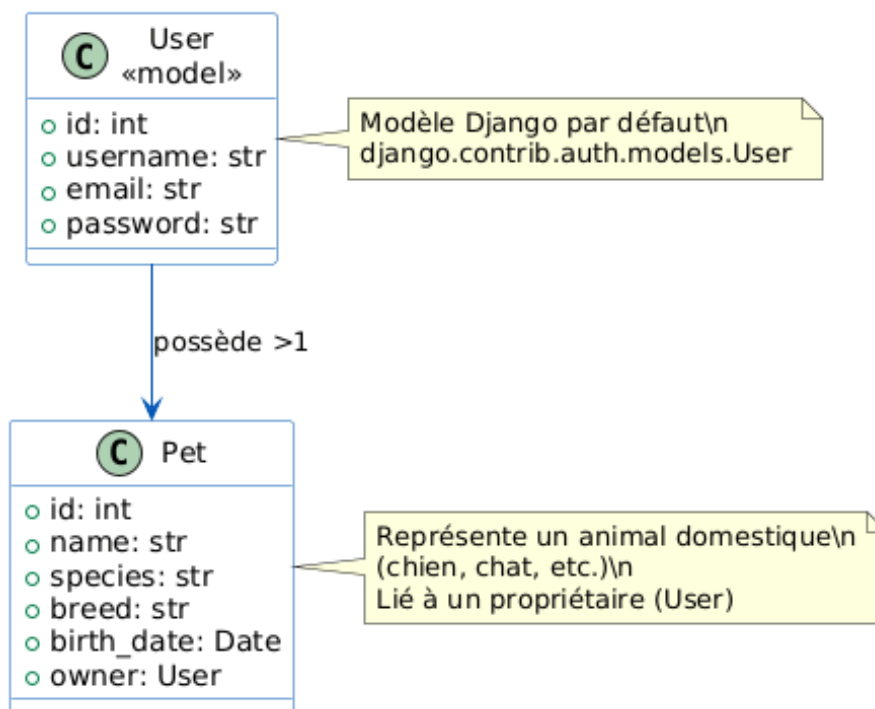


FIGURE 4.6 – Diagramme de Classes - Sprint 3 : Gestion des Animaux

Ce diagramme montre qu'un utilisateur peut avoir un ou plusieurs animaux. Chaque animal (comme un chien ou un chat) est enregistré avec son nom, son espèce, sa race et sa date de naissance, et est lié à son propriétaire (l'utilisateur). Ainsi, chaque personne voit seulement ses propres animaux, ce qui rend le système simple, personnel et sécurisé.

4.3 sprint 4 : Prise de rendez-vous

1. Objectifs du sprint 4

Le Sprint 4 a pour objectif de permettre aux utilisateurs de prendre des rendez-vous vétérinaires en ligne de manière simple et sécurisée. L'utilisateur peut sélectionner l'un de ses animaux, choisir un service (comme une vaccination ou un contrôle), puis réserver une date et une heure disponibles. Le système vérifie automatiquement que le créneau n'est pas déjà pris, lie le rendez-vous à son compte et l'envoie en attente de validation à l'administrateur. Une confirmation est affichée à l'écran, et un email peut être envoyé. Ce sprint rend le système pleinement fonctionnel, en connectant la gestion des animaux à la prise de rendez-vous, offrant ainsi une expérience complète et fluide.

4.3.1 Backlog Produit

Le tableau 2.1 présente le backlog du produit de notre projet, qui détaille les différentes user stories. Il est important de souligner que le backlog du produit est un outil essentiel pour la planification et la gestion des exigences, car il regroupe toutes les fonctionnalités et les tâches à réaliser pour le produit

Tâche	Priorité
Créer le modèle <code>Appointment</code> avec les champs : animal, service, date, heure, téléphone, notes, statut	Haute
Définir les services disponibles (ex : vaccination, contrôle, urgence)	Moyenne
Implémenter les créneaux horaires fixes (ex : 9h00, 9h30, etc.)	Moyenne
Créer le formulaire <code>AppointmentForm</code> basé sur le modèle	Moyenne
Lier le RDV à l'utilisateur connecté (<code>request.user</code>)	Haute
Ajouter la vérification des disponibilités (éviter les doublons)	Haute
Créer la vue <code>appointment_create</code> (POST/GET)	Moyenne
Créer le template <code>create.html</code> avec Bootstrap	Moyenne
Afficher uniquement les animaux de l'utilisateur dans le formulaire	Haute
Envoyer un email automatique après soumission du RDV	Moyenne
Créer la vue <code>appointment_list</code> pour afficher les RDV de l'utilisateur	Moyenne
Ajouter un statut : « En attente », « Confirmé », « Refusé »	Haute
Rediriger vers la liste après création	Basse
Gérer les erreurs (date passée, champ manquant)	Moyenne

TABLE 4.6 – Backlog du Sprint 3

4.3.2 Spécification et conception du sprint

Dans cette section, nous allons présenter la spécification et l'analyse du sprint, en mettant en évidence les différents aspects du développement. Nous aborderons notamment le diagramme de cas d'utilisation, ainsi que les modèles statique et dynamique associés à ce sprint.

Diagramme de cas d'utilisation

La figure 4.7 représente le diagramme de cas d'utilisation de ce sprint.

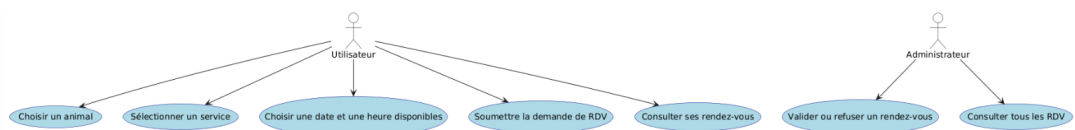


FIGURE 4.7 – Diagramme de cas d'utilisation de sprint 4

Cas d'utilisation	Description
Choisir un animal	L'utilisateur sélectionne l'un de ses animaux enregistrés pour le rendez-vous.
Sélectionner un service	L'utilisateur choisit le type de consultation (ex : vaccination, contrôle).
Choisir une date et une heure disponibles	Le système affiche les créneaux libres ; l'utilisateur en sélectionne un.
Soumettre la demande de RDV	Après validation des données, la demande est envoyée avec statut « En attente ».
Consulter ses rendez-vous	L'utilisateur voit la liste de ses demandes avec leurs statuts (en attente, confirmé, etc.).
Valider ou refuser un rendez-vous	L'administrateur accepte ou rejette la demande depuis l'interface d'administration.

TABLE 4.7 – Description Cas d'utilisation Sprint (4)

4.4 Conclusion

Ce chapitre présente les travaux des sprints 3 et 4.

Le sprint 3 a permis d'ajouter la gestion des animaux : les utilisateurs peuvent désormais ajouter, modifier, supprimer et consulter leurs animaux. Le système vérifie que la date de naissance est valide et que chacun ne voit que ses propres animaux. Un lien vers "Mes animaux" a aussi été ajouté au menu.

Le sprint 4 a introduit la prise de rendez-vous vétérinaires. L'utilisateur choisit un de ses animaux, un service (comme la vaccination), et un créneau horaire libre. La demande est envoyée avec le statut « En attente », et l'administrateur peut la confirmer ou la refuser. Le système empêche les doublons et affiche clairement les rendez-vous de chaque utilisateur.

Ces deux sprints rendent l'application utile et complète, en reliant la gestion des animaux à la réservation de rendez-vous

Chapitre 5

Présentation des interfaces et fonctionnalités de l'application

Introduction

Ce chapitre présente l'interface graphique et les principales fonctionnalités de l'application web développée. Des captures d'écran illustrent le fonctionnement de chaque module et démontrent la convivialité de l'interface utilisateur.

5.1 Page d'accueil

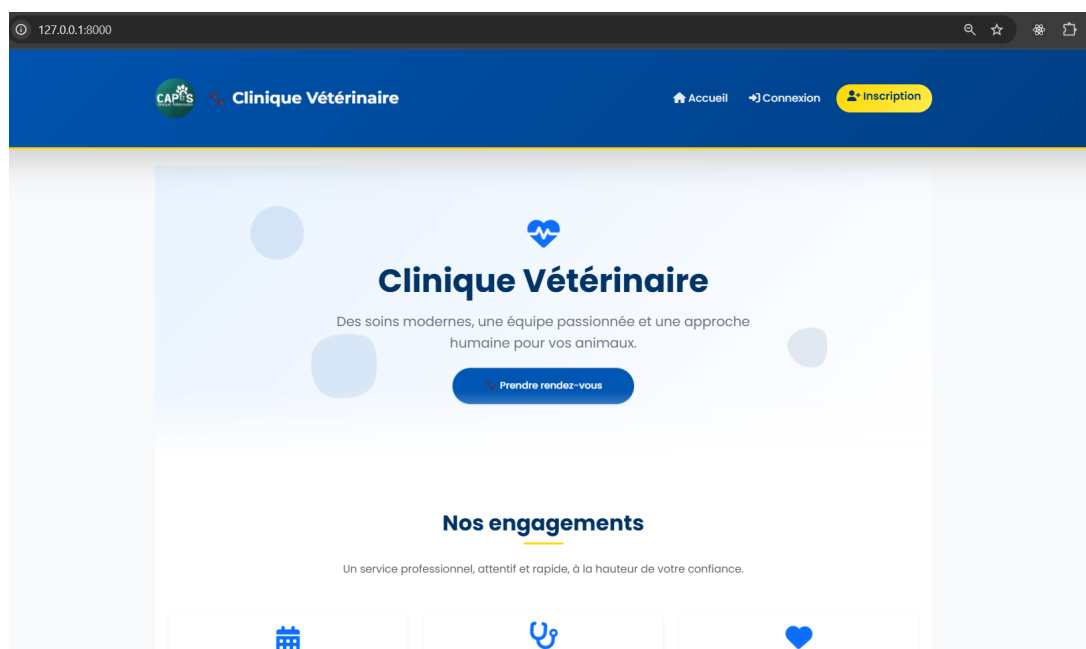
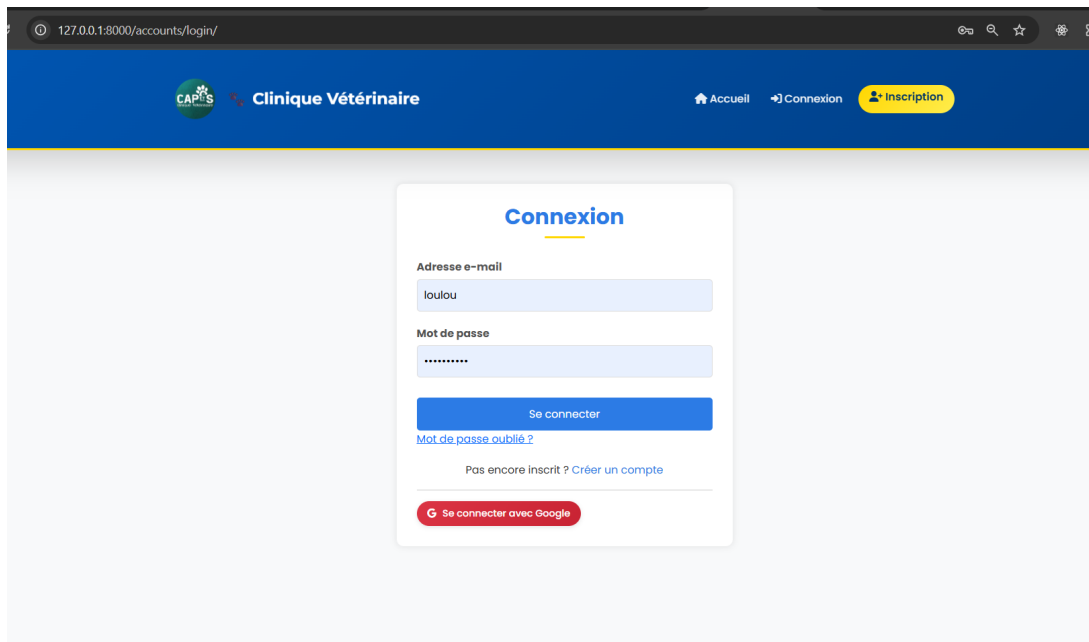


FIGURE 5.1 – Page d'accueil

Description

La page d'accueil présente un aperçu général de la clinique vétérinaire avec des informations sur les services proposés, les horaires d'ouverture et un bouton d'accès à la prise de rendez-vous. Une barre de navigation permet d'accéder aux différentes sections du site.

5.2 Page d'authentification (connexion / inscription)



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/accounts/login/'. The page has a dark blue header with the 'CAPS' logo and 'Clinique Vétérinaire' text on the left. On the right, there are navigation links: 'Accueil', 'Connexion', and a yellow 'Inscription' button. The main content area is light gray and features a white login form titled 'Connexion'. The form includes two input fields: 'Adresse e-mail' with the value 'loulou' and 'Mot de passe' with masked characters. Below these is a blue 'Se connecter' button, a link for 'Mot de passe oublié ?', and a text link 'Pas encore inscrit ? Créer un compte'. At the bottom of the form is a red button with the Google logo and the text 'Se connecter avec Google'.

FIGURE 5.2 – Page de connexion

The screenshot shows the 'Créer un compte' page of the Clinique Vétérinaire website. The header is dark blue with the CAPES logo and 'Clinique Vétérinaire' text on the left, and navigation links 'Accueil', 'Connexion', and 'Inscription' on the right. The main content area is light gray. A white card in the center contains the registration form. The form has four sections: 'Nom d'utilisateur' with a text input field containing 'Choisissez un nom d'utilisateur'; 'Adresse email' with a text input field containing 'Email address'; 'Mot de passe' with a text input field containing 'Password' and a note 'Minimum 10 caractères, évitez les mots de passe trop courants.'; and 'Confirmer le mot de passe' with a text input field containing 'Password (again)'. Below the form is a blue 'S'inscrire' button and a link 'Déjà un compte ? Connectez-vous ici'.

Créer un compte

Nom d'utilisateur

Choisissez un nom d'utilisateur

Adresse email

Email address

Mot de passe

Password

Minimum 10 caractères, évitez les mots de passe trop courants.

Confirmer le mot de passe

Password (again)

S'inscrire

Déjà un compte ? [Connectez-vous ici](#)

FIGURE 5.3 – Page d'inscription

The screenshot shows the 'Mot de passe oublié ?' page of the Clinique Vétérinaire website. The header is dark blue with the CAPES logo and 'Clinique Vétérinaire' text on the left, and navigation links 'Connexion' and 'Inscription' on the right. The main content area is light gray. A white card in the center contains the password reset form. The form has a title 'Mot de passe oublié ?', a paragraph 'Entrez votre adresse e-mail, nous vous enverrons un lien pour réinitialiser votre mot de passe.', a section 'Adresse e-mail' with a text input field containing 'jabloun.omaïma5102000@gmail.com', a blue 'Envoyer le lien' button, and a link '← Retour à la connexion'.

Mot de passe oublié ?

Entrez votre adresse e-mail, nous vous enverrons un lien pour réinitialiser votre mot de passe.

Adresse e-mail

jabloun.omaïma5102000@gmail.com

Envoyer le lien

[← Retour à la connexion](#)

FIGURE 5.4 – Page mot de passe oublié

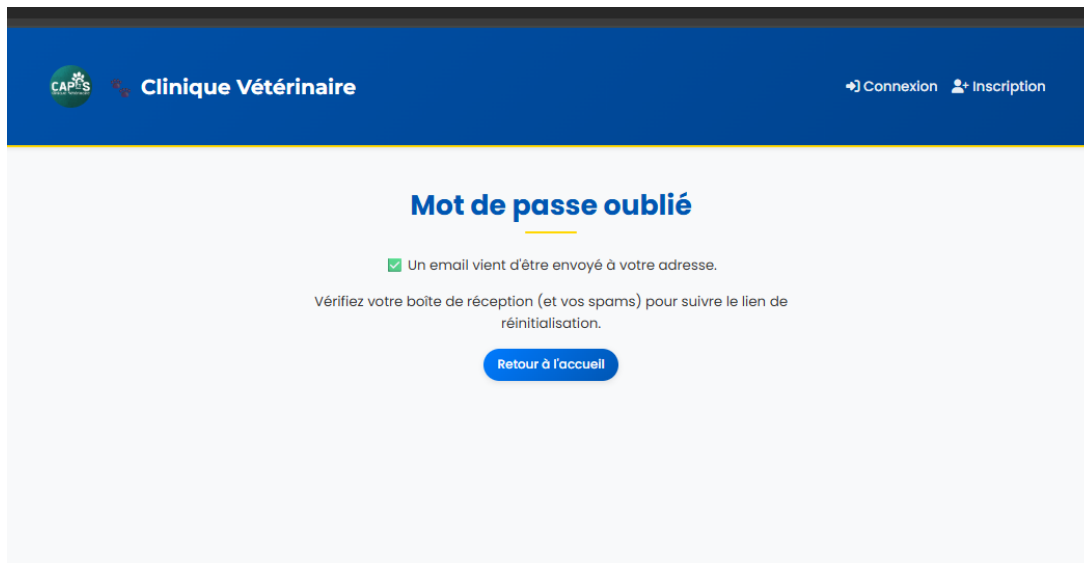


FIGURE 5.5 – Page mot de passe oublié

Description

Cette interface permet aux utilisateurs de s'authentifier ou de créer un nouveau compte. La validation des champs de saisie est assurée à la fois côté client (via JavaScript) et côté serveur (pour garantir la sécurité et l'intégrité des données). En cas d'identifiants incorrects (adresse e-mail ou mot de passe invalide), un message d'erreur approprié est affiché.

Un bouton « Se connecter avec Google » permet une authentification via OAuth 2.0 en utilisant un compte Gmail existant.

Lorsqu'un utilisateur clique sur le lien « Mot de passe oublié ? », il est redirigé vers une page dédiée contenant un champ de saisie pour son adresse e-mail. Après avoir entré son adresse, il peut cliquer sur le bouton « Envoyer le lien ». Le système vérifie alors la validité de l'e-mail (côté serveur) et, si celui-ci correspond à un compte existant, envoie un lien de réinitialisation sécurisé et temporaire à l'adresse fournie.

Un e-mail contenant un lien de réinitialisation vient d'être envoyé à votre adresse. Veuillez consulter votre boîte de réception (et vos courriers indésirables, si nécessaire) et cliquer sur le lien pour définir un nouveau mot de passe.

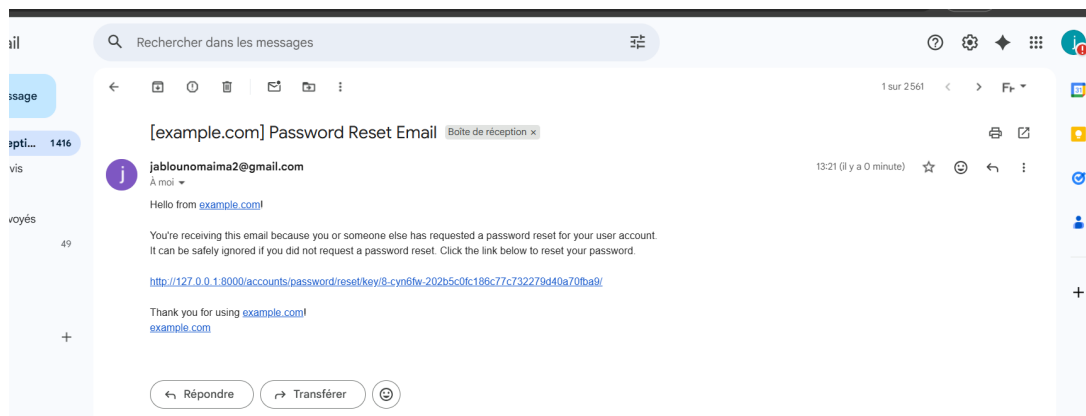


FIGURE 5.6 – Un e-mail contenant un lien de réinitialisation

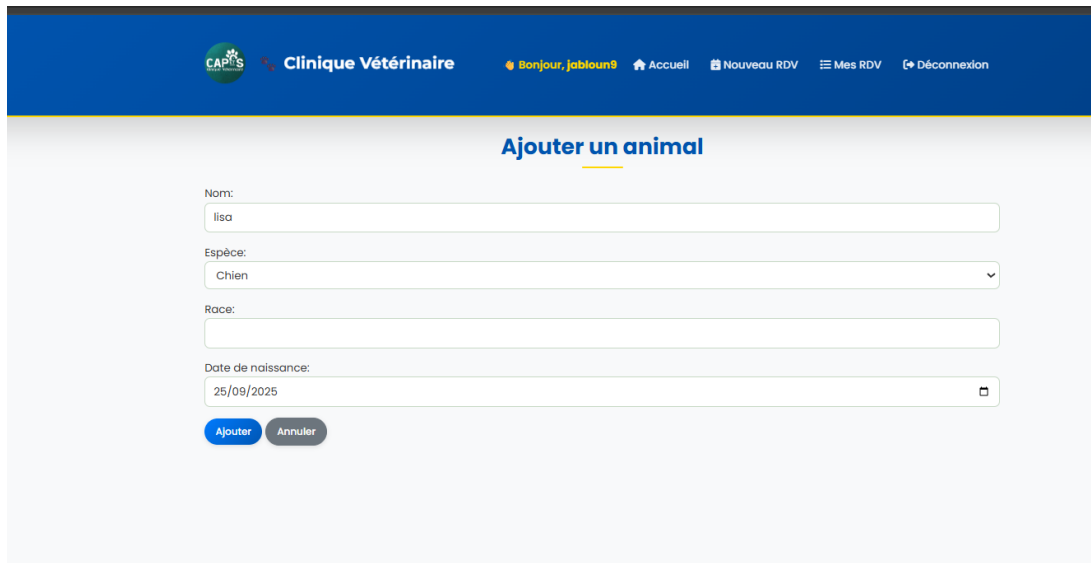
FIGURE 5.7 – Page Réinitialiser le mot de passe

5.3 Processus de Création de Nouveau Rendez-vous

Description

Pour créer un nouveau rendez-vous, l'utilisateur accède à un formulaire qui se déroule en plusieurs étapes séquentielles.

Étape 1 – Ajout d’un animal



The screenshot shows a web interface for a veterinary clinic. At the top is a dark blue header with the logo 'CAPES Clinique Vétérinaire' on the left. To the right of the logo, it says 'Bonjour, jabilou9'. Further right are links: 'Accueil', 'Nouveau RDV', 'Mes RDV', and 'Déconnexion'. Below the header, the main content area has a title 'Ajouter un animal' in blue. Underneath the title is a form with four fields: 'Nom:' with the value 'lisa', 'Espèce:' with a dropdown menu showing 'Chien', 'Race:' with an empty text box, and 'Date de naissance:' with a date picker showing '25/09/2025'. At the bottom of the form are two buttons: 'Ajouter' (blue) and 'Annuler' (grey).

FIGURE 5.8 – Page Réinitialiser le mot de passe

L'utilisateur a la possibilité de cliquer sur le bouton « **Ajouter un animal** », ce qui déclenche l'ouverture d'un sous-formulaire modal ou d'une section dédiée, permettant de renseigner les informations obligatoires suivantes :

- Nom de l'animal
- Espèce
- Race
- Date de naissance

Après validation de ce sous-formulaire, l'entité *Animal* est enregistrée (de manière temporaire ou permanente dans le profil utilisateur, selon l'implémentation technique choisie) et devient immédiatement sélectionnable pour la suite du processus de réservation.

Étape 2 – Saisie des détails du rendez-vous

FIGURE 5.9 – Page Réinitialiser le mot de passe

Dans le formulaire principal intitulé « **Nouveau rendez-vous** », l'utilisateur doit spécifier les détails de la réservation en sélectionnant ou en saisissant les éléments suivants :

- L'animal concerné (sélectionné parmi la liste des animaux déjà enregistrés et/ou ajoutés à l'Étape 1).
- Le service souhaité (e.g., consultation, vaccination, chirurgie mineure, etc.).
- La date et l'heure du rendez-vous (en tenant compte des disponibilités).
- Son numéro de téléphone de contact.
- Une remarque facultative (champ texte libre).

Une fois que toutes les informations requises sont complétées et validées, l'utilisateur clique sur le bouton « **Réserver** » afin de confirmer et de finaliser la création du rendez-vous dans le système.

Gestion et Suivi des Rendez-vous - Vue Utilisateur

Une fois la demande de rendez-vous , son statut et sa gestion sont régis par les règles suivantes :

Affichage Initial et Statut Après sa création, le rendez-vous est immédiatement intégré à la liste « **Mes Rendez-vous** » de l'utilisateur. Il se voit attribuer le statut initial « **En attente** », signalant que la validation par l'administrateur est requise.

Détails Affichés dans le Tableau Le tableau de suivi présente les informations clés de la réservation :

- L'Animal concerné.
- Le Type de service demandé (e.g., Vaccination).
- La Date et l'Heure demandées.
- Le Numéro de téléphone de contact fourni.
- Le Statut actuel : « **En attente** ».

Action Utilisateur : Annulation Tant que le statut demeure « **En attente** », l'utilisateur dispose du bouton d'action « **Annuler** » pour révoquer la demande.

Scénario de Refus par l'Administrateur Si l'administrateur refuse la demande (✗) :

- Le statut est mis à jour vers « **Refusé** » (ou son équivalent implémenté).
- Un message ou une remarque explicative, saisie par l'administrateur, est affiché à l'utilisateur.
- L'utilisateur conserve la possibilité de **reprogrammer le rendez-vous** en modifiant la date et/ou l'heure, mettant à jour les autres champs si nécessaire, puis en soumettant à nouveau la demande.

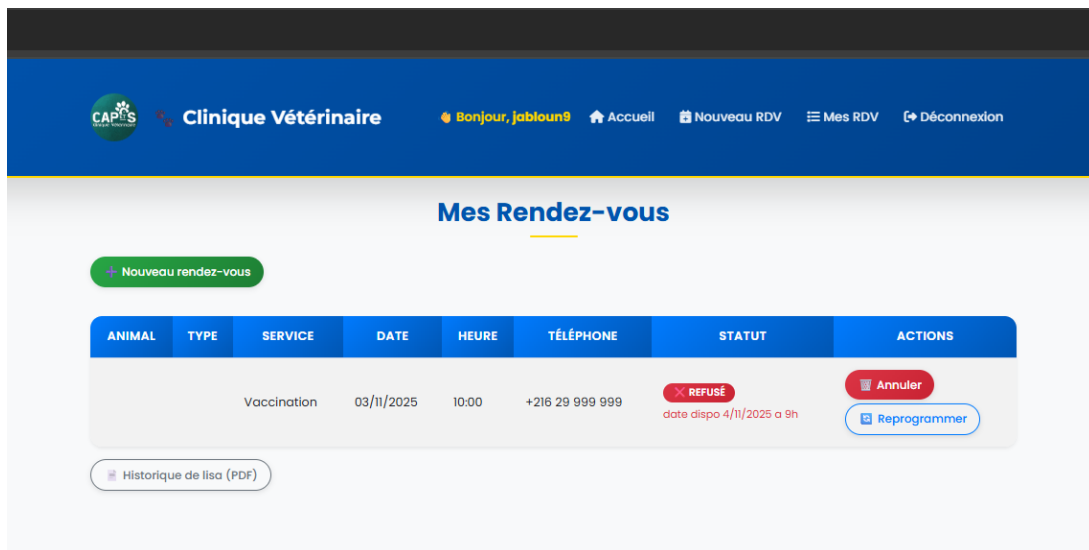


FIGURE 5.10 – page d'utilisateur après le Refus de RDV par l'Administrateur

Scénario de Confirmation par l'Administrateur Si l'administrateur valide la demande (✓) :

- Le statut passe automatiquement à « **Confirmé** ».
- Le rendez-vous est alors considéré comme validé et est figé dans le calendrier, sous réserve d'annulations ou de modifications futures gérées par l'administrateur ou selon les règles métier spécifiques.

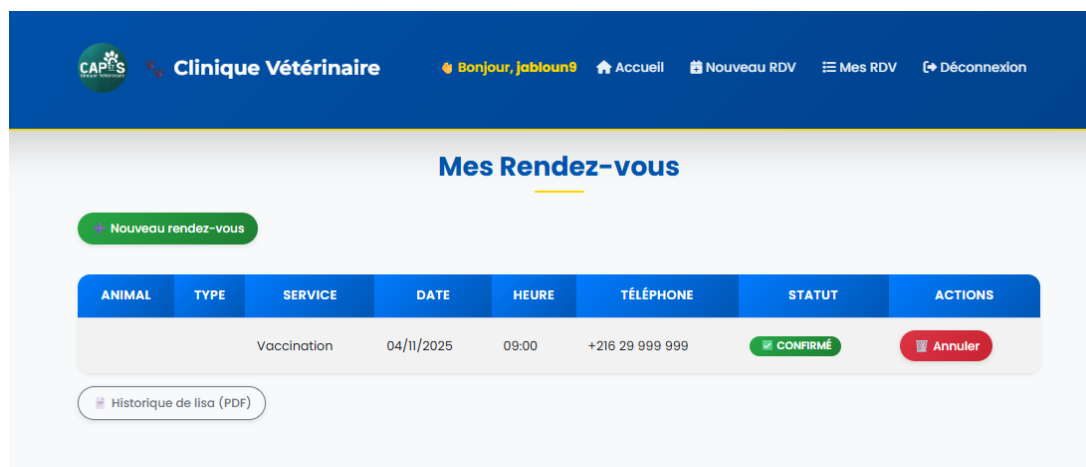


FIGURE 5.11 – page d'utilisateur après le Confirmation de RDV par l'Administrateur

Historique Vétérinaire Détaillé par Animal

La gestion des données médicales est structurée de manière individuelle. Chaque entité *Animal* associée au profil utilisateur possède son propre historique médical, accessible via l'interface et téléchargeable.

Accessibilité et Format L'historique est accessible spécifiquement pour chaque animal et peut être exporté au format **PDF** pour archivage.

Contenu du Rapport PDF Le document généré est un récapitulatif exhaustif qui regroupe les éléments suivants :

- Les métadonnées d'identification de l'impression : la date de génération du document et le nom du propriétaire.
- L'intégralité des rendez-vous ayant un statut **Confirmé** ou **Réalisé**, présentés sous forme de tableau détaillant :
 - Le service effectué.
 - La date de la prestation.
 - Le statut final du rendez-vous.
- Les notes spécifiques rédigées par le client lors de la prise de rendez-vous ou de la consultation.
- Les notes cliniques et observations rédigées par le vétérinaire après l'acte.

Principe de Séparation des Données Le principe de **Un historique = Un animal** garantit un suivi médical personnalisé, clair et sécurisé, évitant toute confusion entre les dossiers des différents compagnons de l'utilisateur.

Historique vétérinaire

Animal : lisa (Chien)

Propriétaire : jabloun omaima

Date d'impression : 02/11/2025 12:05

Service	Date	Statut	Notes du client	Notes Vétérinaire
Vaccination	20/11/2025	Confirmé	Mon chien a des puces.	nexgard
Vaccination	04/11/2025	Confirmé	nexgard	-

FIGURE 5.12 – historique de RDV en pdf

5.4 Interface Administrateur

L'administrateur peut se connecter à l'interface d'administration en utilisant l'adresse e-mail admin@gmail.com et son mot de passe associé.

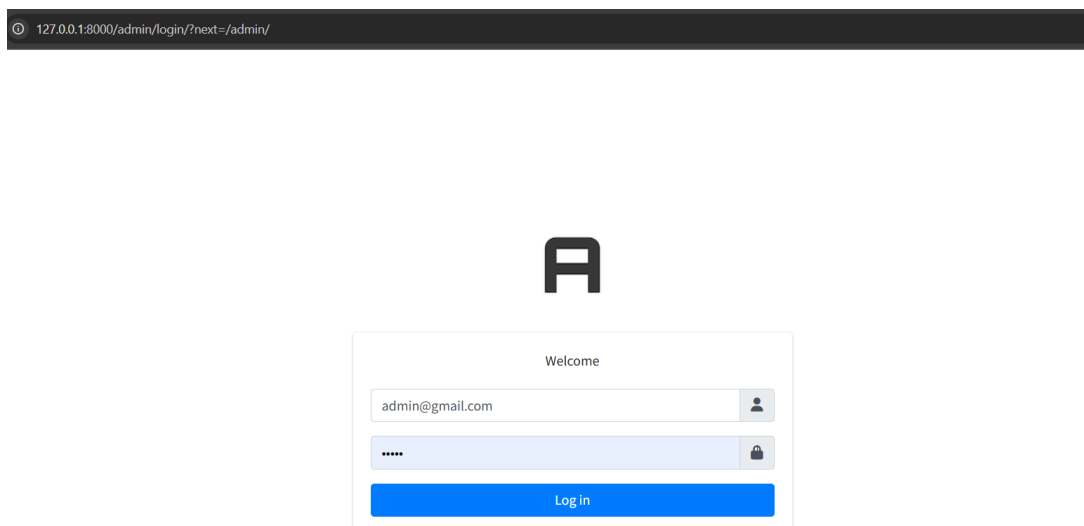


FIGURE 5.13 – interface d’admin

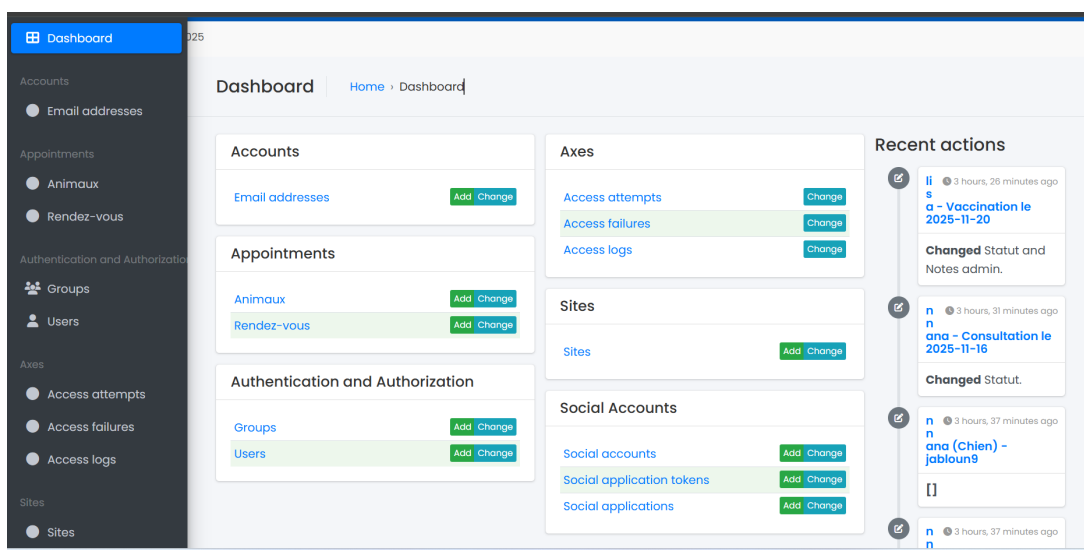


FIGURE 5.14 – interface d’admin

Gestion des Adresses E-mail – Interface d’Administration Django

Cette interface représente la vue de gestion du modèle *EmailAddress* au sein du panneau d’administration de Django. Elle fournit un accès direct à la liste complète des adresses électroniques associées aux comptes des utilisateurs de la clinique vétérinaire.

Vue d’Ensemble des Données

- **Entité Managée :** Modèle de données *EmailAddress* (table dédiée à la persistance des e-mails utilisateur).

- **Volumétrie (Exemple) :** Deux (2) enregistrements sont actuellement affichés, comme indiqué au bas de la liste.
- **Champs (Colonnes) Visibles :**
 - **EMAIL ADDRESS :** Affiche l'adresse électronique complète de l'utilisateur (*e.g.*, JABLOUN.OMAIMA5102000@GMAIL.COM).
 - **USER :** Affiche le nom d'utilisateur associé au compte lié à l'adresse e-mail (*e.g.*, jabloun9).
 - **PRIMARY :** Indicateur booléen () spécifiant si l'e-mail est défini comme l'adresse principale du compte.
 - **VERIFIED :** Indicateur booléen () spécifiant si l'adresse a été validée par un processus de vérification (souvent par lien cliquable).

Fonctionnalités d'Administration L'interface standard de l'administration Django met à disposition les outils suivants pour la manipulation des données :

- **Filtres :** Panneau latéral permettant de filtrer la liste en fonction des attributs booléens (*Primary* et *Verified*).
- **Recherche :** Un champ de recherche textuelle est disponible pour interroger la base de données sur des adresses e-mail ou des noms d'utilisateur spécifiques.
- **Actions Groupées :** Mécanisme de sélection multiple pour appliquer des actions prédéfinies à plusieurs enregistrements simultanément.
- **Ajout Manuel :** Le bouton « Add email address » permet la création manuelle d'une nouvelle entrée dans la table, utile pour la gestion des cas exceptionnels ou les tests par l'administrateur.

Gestion Détaillée des Fiches Animales – Vue Administrateur

The screenshot displays the 'Gestion Détaillée des Fiches Animales – Vue Administrateur' interface. On the left is a dark sidebar with a menu containing sections like 'Accounts', 'Appointments', 'Authentication and Authorization', 'Axes', and 'Sites'. The 'Appointments' section is active, with 'Animaux' selected. The main content area has two tabs: 'General' (active) and 'Rendez-vous'. The 'General' tab shows a form for an animal named 'lisa'. The form includes fields for 'Nom' (lisa), 'Owner' (jabloun9), 'Espèce' (Chien), 'Race' (empty), and 'Date de naissance' (2025-09-25). Below the date field, there is a 'Today' button and a note: 'Note: You are 1 hour ahead of server time.' The 'Created at' field shows 'Nov. 2, 2025, 11:03 a.m.'. At the bottom of the form, there are three buttons: 'Export PDF', 'TÉLÉCHARGER L'HISTORIQUE (PDF)', and 'Delete'. A green 'SAVE' button is also present.

FIGURE 5.15 – interface d'admin

L'interface d'administration offre une vue et un contrôle détaillés sur les fiches de chaque animal enregistré, permettant une gestion complète des dossiers patients.

1. Formulaire de Modification de la Fiche Animal La première vue correspond au formulaire de modification d'une entité *Animal* (Exemple : **Lisa**, chien, propriétaire **jabloun9**). Ce formulaire permet à l'administrateur de :

- **Consulter et modifier** les informations primaires : le Nom, l'Espèce, la Race et la Date de naissance de l'animal.
- **Visualiser** les métadonnées techniques, notamment la Date de création du dossier.
- **Gérer les actions de maintenance** :
 - Sauvegarder les modifications apportées (« Save »).
 - Supprimer définitivement le dossier de l'animal.
 - Exporter l'historique médical complet de l'animal au format **PDF** (fonctionnalité de traçabilité).

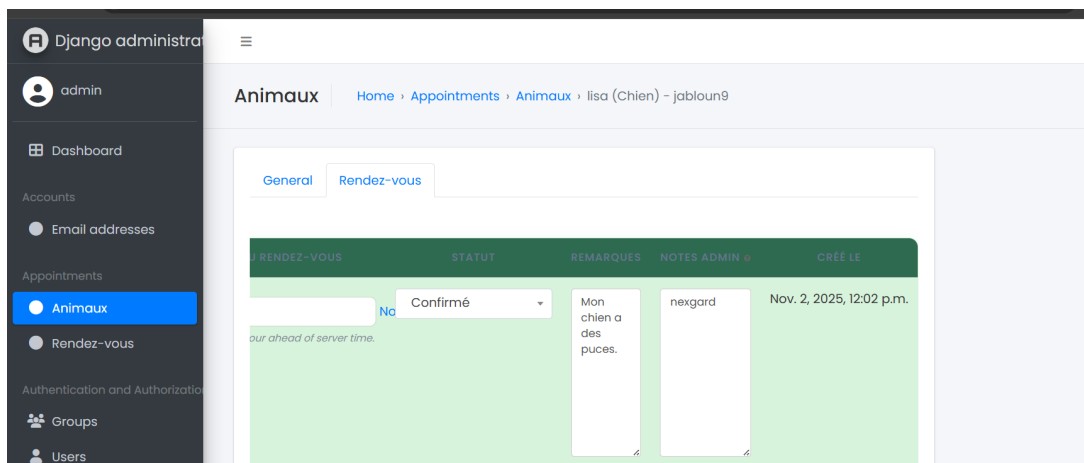


FIGURE 5.16 – interface d’admin

2. Onglet des Rendez-vous Associés La seconde image montre l’onglet « **Rendez-vous** » intégré à la fiche de l’animal, offrant une vue centralisée des interactions passées et futures :

- **Liste des RDV** : Affichage de tous les rendez-vous associés à cet animal, incluant ceux avec un statut « **Confirmé** ».
- **Détails des Interactions** : Pour chaque rendez-vous, le tableau affiche :
 - Le service concerné.
 - La date de la prestation.
 - Le statut actuel du RDV.
 - Les remarques saisies par le client (*e.g.*, « Mon chien a des puces. »).
 - Les notes administratives ou vétérinaires internes (*e.g.*, « nexgard »).
 - La date de création du rendez-vous.

Conclusion de la Vue Administrateur Cette vue centralisée est essentielle pour l’administrateur, car elle permet un **suivi efficace et rapide** des soins prodigués, garantissant une **traçabilité complète** de toutes les interventions et des échanges avec le propriétaire.

Gestion des rendez-vous (RDV) – Vue Administrateur

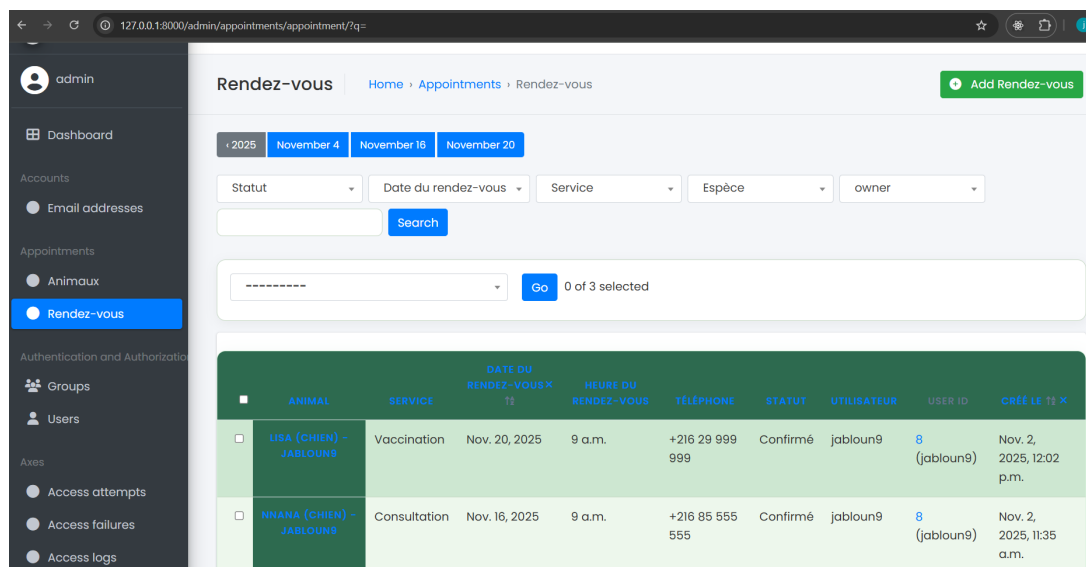


FIGURE 5.17 – interface d’admin

Cette vue représente l’interface d’administration du modèle de données *RendezVous* (RDV). Elle est l’outil central pour l’administrateur afin de surveiller, valider et gérer l’ensemble des réservations effectuées à la clinique.

Détails Affichés dans la Liste Chaque enregistrement de rendez-vous est présenté avec un ensemble de détails clés, facilitant une compréhension rapide de la demande :

- **Identification de l’Animal** : Nom de l’animal, son espèce, et le nom du propriétaire (liaison vers l’entité *Animal*).
- **Service** : Le type de prestation demandée (*e.g.*, Vaccination, Consultation).
- **Planification** : La Date et l’Heure précises du RDV.
- **Contact** : Le Numéro de téléphone fourni par l’utilisateur.
- **Statut** : L’état actuel du rendez-vous (*e.g.*, « En attente », « Confirmé », « Refusé »).
- **Traçabilité** : L’Utilisateur associé (nom d’utilisateur et ID) et la Date de création du RDV.

Fonctionnalités de Gestion et de Filtrage L’interface est optimisée pour la gestion des données grâce aux outils suivants :

- **Filtres Avancés** : Disponibilité de filtres pour segmenter rapidement la liste selon des critères essentiels : le **Statut** du RDV, le **Mois** de la réservation, le **Service** demandé, l’**Espèce** de l’animal et le **Propriétaire** concerné.
- **Actions Groupées** : Système de sélection multiple permettant à l’administrateur d’appliquer des actions de masse (*e.g.*, confirmation ou refus groupé) via le bouton « Go ».

Rôle Fonctionnel Cette interface centralisée permet à l’administrateur d’assurer une **traçabilité complète** et une gestion proactive de l’agenda de la clinique, garantissant la validation des RDV et la bonne organisation des services.

Gestion des Users – Vue Administrateur

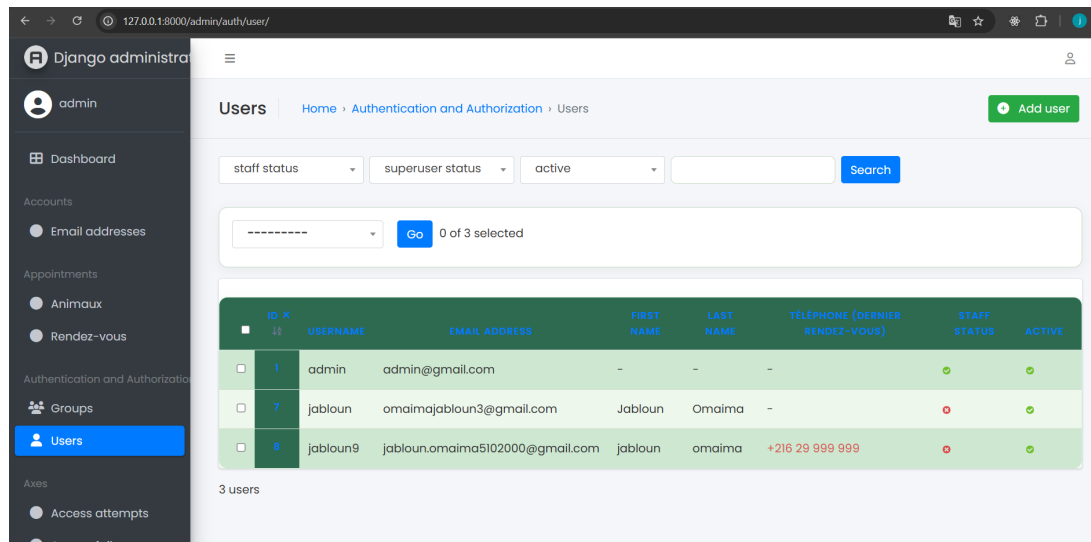


FIGURE 5.18 – interface d’admin

La page « Users » au sein de l’interface d’administration Django est l’outil principal permettant à l’administrateur de gérer les comptes et les accès des utilisateurs inscrits sur la plateforme de la clinique.

Liste et Informations des Utilisateurs La liste affiche un aperçu des utilisateurs enregistrés (Exemple : 3 utilisateurs affichés) avec les colonnes d’information suivantes :

- **Identification** : L’ID unique et le Nom d’utilisateur.
- **Coordonnées** : L’Adresse e-mail, ainsi que le Prénom et le Nom (s’ils ont été renseignés par l’utilisateur).
- **Contact et Historique** : Le Numéro de téléphone de contact et, si la fonctionnalité est implémentée, une indication du dernier rendez-vous pris par cet utilisateur.
- **Statut d’Accès (*Permissions*)** :
 - **Statut Staff** : Indique si l’utilisateur possède les droits d’accès au panneau d’administration (*admin access*).
 - **Statut Active** : Indique si le compte est actif ou s’il a été désactivé.

Fonctionnalités de l’Administrateur L’administrateur dispose des actions de gestion suivantes :

- **Création de Compte** : Ajout d’un nouvel utilisateur via le bouton « Add user » (utile pour la création manuelle de comptes staff ou clients).
- **Filtrage** : Possibilité de filtrer la liste des utilisateurs selon leurs statuts de permission (*Staff status*, *Superuser status*) ou l’état de leur compte (*Active status*).

- **Actions Groupées** : Outil de sélection multiple pour appliquer des modifications de masse aux comptes sélectionnés (*e.g.*, désactiver plusieurs comptes simultanément).

Surveillance des Tentatives d'Accès – Module de Sécurité (*Access attempts*)

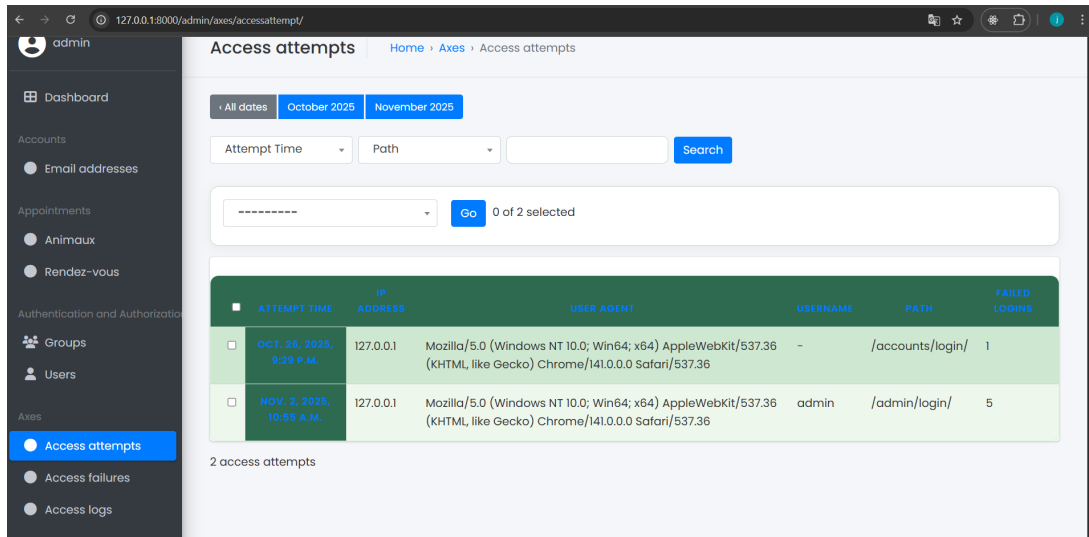


FIGURE 5.19 – interface d’admin

Cette interface d’administration, gérée par le module de sécurité **Django Axes**, est dédiée à la surveillance et à l’analyse des tentatives de connexion au système, incluant les accès réussis et les échecs.

Rôle Sécuritaire Cette vue est essentielle pour la sécurité du site, car elle permet à l’administrateur de **détecter rapidement toute activité suspecte** ou des schémas de connexion anormaux, y compris les tentatives d’attaque par force brute sur les pages sensibles comme le panneau d’administration.

Détails des Tentatives Enregistrées La liste des tentatives d’accès (Exemple : 2 enregistrements) fournit une traçabilité complète incluant les informations suivantes :

- **Horodatage (*Timestamp*)** : La Date et l’Heure précises de la tentative de connexion.
- **Origine** : L’Adresse IP de l’utilisateur (*e.g.*, 127.0.0.1).
- **Environnement** : Le *User Agent* (navigateur et système d’exploitation utilisé par le client).
- **Cible** : Le Nom d’utilisateur ciblé (si renseigné, *e.g.*, admin) et le Chemin d’accès à la page de connexion (*e.g.*, /admin/login/).
- **Échec** : Le Nombre de connexions échouées (*Failed logins*) associées à cette tentative ou cette source.

Outils d’Analyse et de Gestion L’administrateur dispose d’outils pour l’investigation et la maintenance :

- **Filtres** : Filtres contextuels par période (Mois, Heure) et par **Chemin d'accès**, permettant d'isoler rapidement les incidents.
- **Actions Groupées** : Système de sélection multiple pour appliquer des actions de maintenance (telles que le déblocage d'une IP ou la suppression d'une tentative) sur plusieurs enregistrements.

Surveillance des Échecs d'Accès – Verrouillage de Sécurité (*Access failures*)

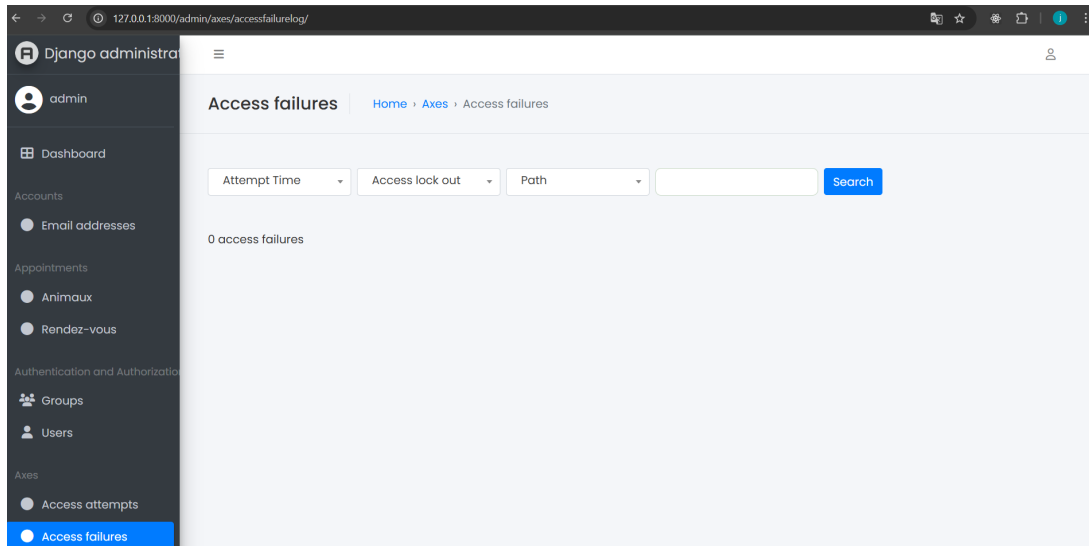


FIGURE 5.20 – interface d'admin

Cette vue, complémentaire à la gestion des tentatives d'accès (section 5.4), affiche la liste des échecs de connexion ayant déclenché un mécanisme de sécurité, tel que le verrouillage d'un compte ou d'une adresse IP.

Indicateur de Sécurité Le nombre d'enregistrements affichés (*e.g.*, 0 échec d'accès) indique qu'aucune tentative de connexion n'a atteint le seuil de sécurité nécessitant le blocage ou le verrouillage d'un utilisateur ou d'une adresse IP pour le moment.

Rôle Fonctionnel et Sécuritaire

- **Surveillance des Blocages** : Cette page permet à l'administrateur de surveiller spécifiquement si des utilisateurs ont été bloqués (*lock out*) suite à un nombre excessif de tentatives de saisie de mot de passe incorrect (*trop d'essais de mot de passe incorrects*).
- **Maintenance** : Elle est essentielle pour le support utilisateur, permettant d'identifier et de débloquer manuellement des comptes qui auraient été verrouillés par le système de sécurité.

Outils de Filtrage L'administrateur peut utiliser les filtres disponibles pour l'analyse des incidents de sécurité :

- **Heure de la tentative.**

- Statut de verrouillage (*lock out status*).
- Chemin d'accès ciblé par l'échec.

Audit et Traçabilité des Connexions – Journaux d'Accès (*Access logs*)

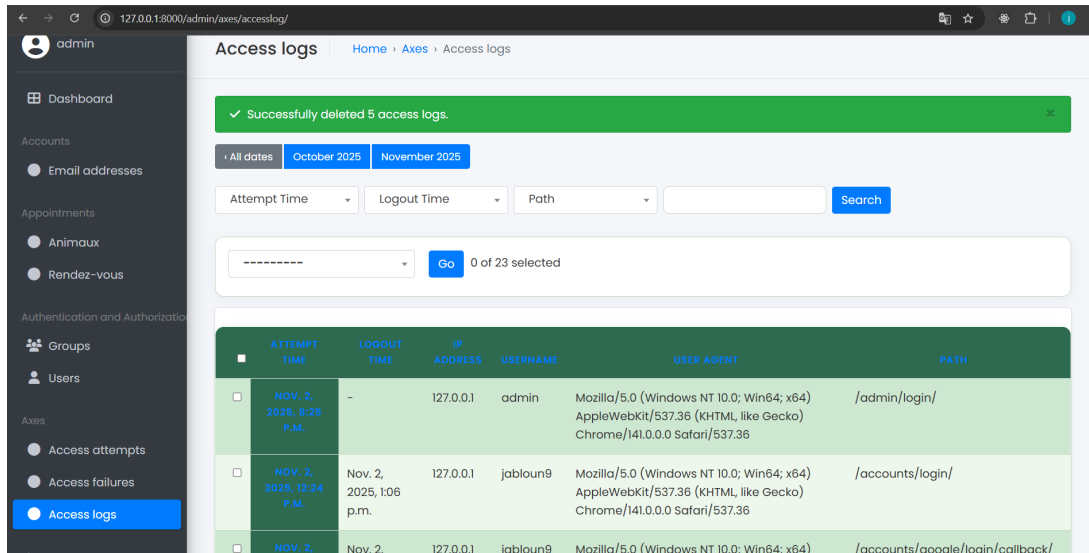


FIGURE 5.21 – Audit et Traçabilité des Connexions

La page « Access logs » fournit à l'administrateur un historique complet des connexions **réussies** à la plateforme. Elle est essentielle pour l'audit et la traçabilité des activités des utilisateurs.

Rôle d'Audit Contrairement aux vues d'échecs, cette interface documente les accès valides, permettant de vérifier la conformité des utilisateurs sans soulever d'alerte de sécurité, et assurant un suivi de l'utilisation du système. Le total des entrées (Exemple : 28 logs) témoigne de l'activité du site.

Détails des Entrées d'Audit Chaque entrée du journal de bord enregistre les métadonnées suivantes :

- **Durée de la Session** : L'Heure de connexion (*Attempt Time*) et, si le mécanisme est implémenté, l'Heure de déconnexion (*Logout Time*).
- **Origine Physique** : L'Adresse IP de la machine cliente (*e.g.*, 127.0.0.1).
- **Identification** : Le Nom d'utilisateur ayant réussi la connexion (*e.g.*, jabloun).
- **Environnement** : Le *User Agent* (navigateur et système d'exploitation utilisé).
- **Point d'Accès** : Le Chemin (*Path*) par lequel la connexion a été initiée (*e.g.*, /accounts/login/).

Fonctionnalités de Filtrage Pour faciliter l'analyse, l'administrateur peut filtrer les journaux par :

- Période (**Mois**, *e.g.*, Octobre / Novembre 2025).
- Heure spécifique.
- Chemin d'accès.

Configuration des Sites – Gestion Multi-Domaines (*Sites*)

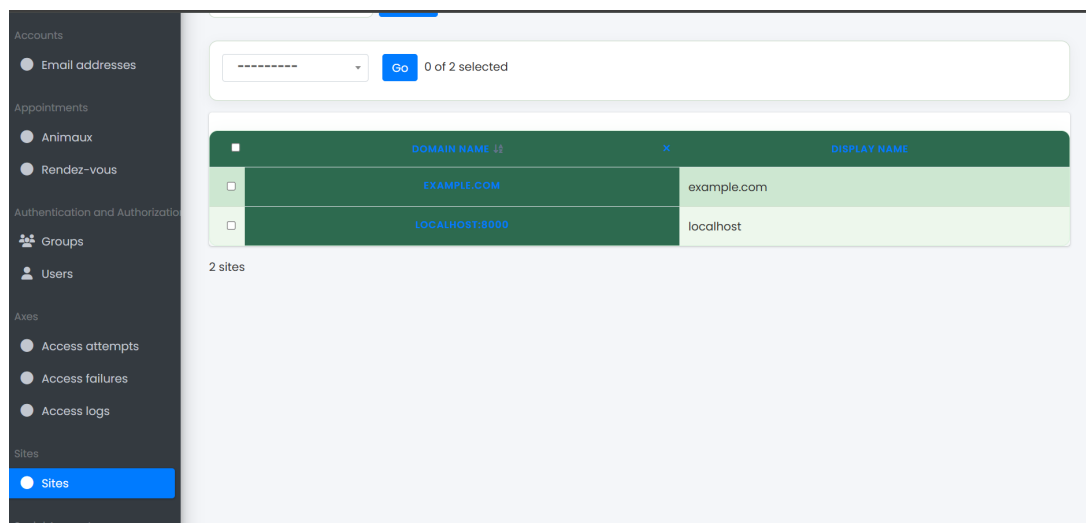


FIGURE 5.22 – Configuration des Sites

Cette interface d'administration affiche la liste des configurations de sites enregistrées au sein de l'application Django (utilisant généralement le framework `django.contrib.sites`). Elle est essentielle pour la gestion des environnements et des domaines multiples.

Rôle Fonctionnel La fonctionnalité de gestion des sites est particulièrement utile dans les scénarios suivants :

- **Environnements Multiples** : Différencier les URL et les paramètres entre l'environnement de développement/test (*e.g.*, `localhost:8000`) et l'environnement de production (*e.g.*, `example.com`).
- **Personnalisation du Contenu** : Adapter le contenu ou les règles métier en fonction du domaine spécifique par lequel l'utilisateur accède à l'application.

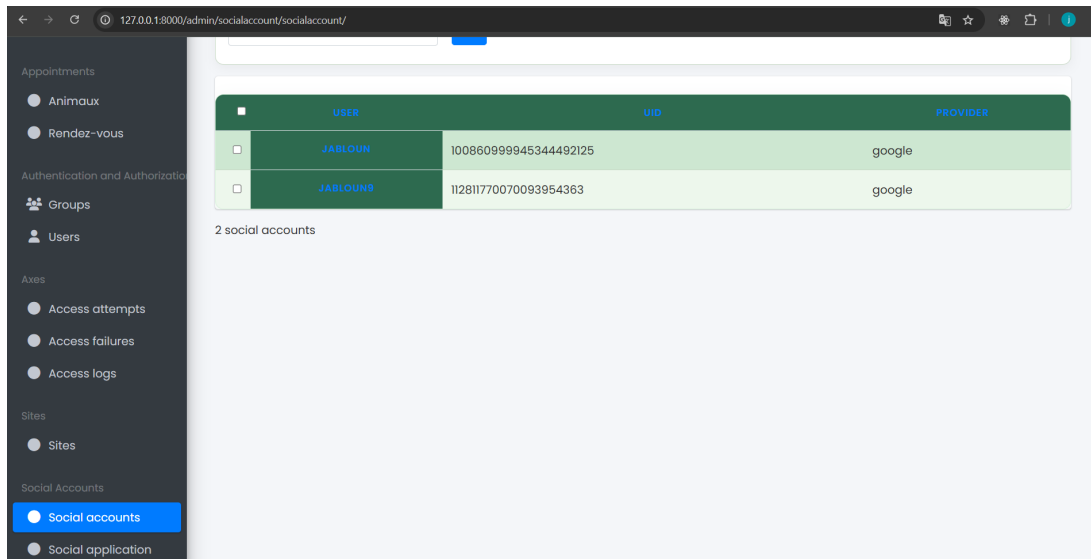
Détails des Configurations Affichées La liste présente les configurations de sites définies (Exemple : 2 sites configurés), détaillant :

- **Domaine** : Le nom de domaine ou l'adresse du site (*e.g.*, `example.com` ou `localhost:8000`).
- **Nom d’Affichage** : Le nom court ou label associé au site.

Actions Administratives L'administrateur peut interagir avec ces configurations :

- **Modification/Suppression** : Chaque site peut être sélectionné pour une modification détaillée de ses paramètres ou pour sa suppression.
- **Actions Groupées** : Le système permet la sélection multiple d'entrées pour des actions de masse (actuellement, 0 sur 2 éléments sont sélectionnés).

Gestion des Comptes Sociaux – Authentification Externe (*Social accounts*)



	USER	UID	PROVIDER
☐	JABLOUN	10086099945344492125	google
☐	JABLOUN9	112811770070093954363	google

2 social accounts

FIGURE 5.23 – Configuration des Sites

Cette interface d’administration affiche l’inventaire des comptes sociaux ayant été utilisés par les utilisateurs pour l’authentification externe, généralement via un mécanisme de type **Single Sign-On (SSO)** basé sur **OAuth 2.0** (ou via une librairie comme Django Allauth).

Rôle de l’Interface Cette vue permet à l’administrateur de maintenir la liste des connexions établies via des fournisseurs tiers (*Social Providers*), assurant la traçabilité des authentifications non basées sur un mot de passe local.

Détails des Connexions Affichées La liste des comptes sociaux connectés (Exemple : 2 comptes) présente les informations d’identification essentielles :

- **Utilisateur Associé** : Le Nom d’utilisateur de l’application lié au compte social (*e.g.*, JABLOUN et JABLOUN9).
- **Fournisseur (*Provider*)** : La source de l’authentification (ici, **Google** pour les deux entrées).
- **Identifiant Unique (UID)** : L’identifiant unique et permanent de l’utilisateur fourni par le fournisseur externe (*e.g.*, 10086099...) – cet UID est la clé de la connexion OAuth.

Actions Administratives L’interface confère à l’administrateur les capacités de gestion suivantes :

- **Surveillance** : Visualiser rapidement quels utilisateurs exploitent l’authentification sociale.
- **Maintenance** : Gérer ou supprimer les connexions sociales si nécessaire (par exemple, en cas de besoin de réinitialisation ou de problème de sécurité).

- **Identification** : Identifier les utilisateurs par leur **UID** pour les besoins d’audit ou de synchronisation.

Configuration des Applications Sociales – Authentification OAuth 2.0

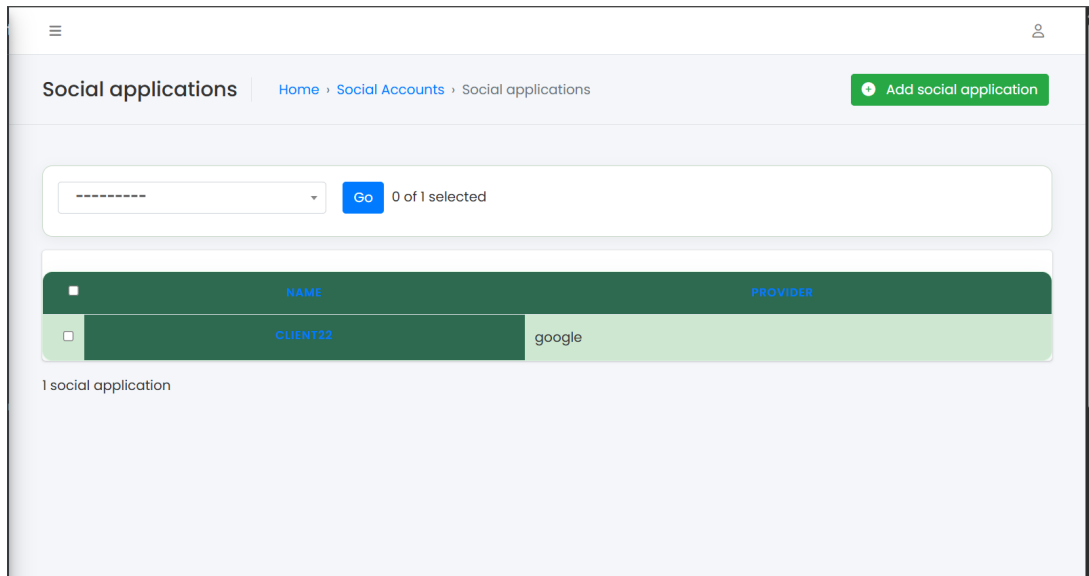


FIGURE 5.24 – Configuration des Sites

Cette interface d’administration (*Social applications*) permet la gestion et la configuration des fournisseurs d’authentification externe qui exploitent le protocole **OAuth 2.0** et **OpenID Connect**. Elle est essentielle pour l’activation du *Single Sign-On (SSO)* via des services tiers.

État de la Configuration La liste affiche l’état actuel de la configuration (Exemple : 1 application sociale configurée).

- **Fournisseur (*Provider*)** : L’application est configurée pour utiliser **Google** comme fournisseur d’identité externe.
- **Nom Interne** : L’application est étiquetée « CLIENT22 », qui correspond généralement à l’identifiant client (*Client ID*) unique fourni par Google lors de l’enregistrement de l’application OAuth.

Rôle Fonctionnel et Sécuritaire

- **Activation de la Connexion Sociale** : Cette configuration est un prérequis technique indispensable pour que l’application puisse déléguer l’authentification à Google et permettre aux utilisateurs de se connecter via leur compte.
- **Sécurité** : Elle assure que le flux d’authentification respecte le protocole **OAuth 2.0**, garantissant une connexion sécurisée sans que le site de la clinique ne stocke les mots de passe des comptes Google.

Extensibilité L’administrateur a la capacité d’**ajouter d’autres fournisseurs** d’identité (Facebook, GitHub, etc.) via le bouton « Add social application », permettant l’évolutivité du système d’authentification.

5.5 Conclusion

Dans ce chapitre, nous avons illustré et documenté les interfaces clés de notre solution, en portant une attention particulière au panneau d’administration, développé avec le framework Django.

Ce panneau d’administration fournit à l’administrateur un ensemble d’outils lui permettant de gérer facilement et de manière centralisée :

- Les fiches détaillées des **utilisateurs** et de leurs **animaux**.
- Le cycle complet des **rendez-vous** (création, confirmation, historique médical).
- La gestion des méthodes de **connexion** (par mot de passe classique ou via l’authentification sociale Google).

De plus, nous avons mis en évidence les outils de **sécurité** intégrés qui permettent de surveiller et de journaliser tous les accès au site. En définitive, cette interface assure une **gestion simple, organisée et sécurisée** de l’ensemble des opérations et des données de la clinique vétérinaire.

5.6 Conclusion Générale

Ce stage a été l’occasion de concevoir et de développer intégralement une application web destinée à la gestion des opérations d’une clinique vétérinaire, en se basant sur la puissance et la robustesse du framework **Django**.

L’application a permis la mise en œuvre de deux espaces fonctionnels distincts :

- **L’Espace Client** : Offrant la possibilité aux utilisateurs de prendre des rendez-vous en ligne, de gérer leur profil animalier et de consulter ou d’exporter l’historique médical de leurs compagnons.
- **L’Espace Administrateur** : Fournissant des outils de gestion centralisée pour les comptes utilisateurs, les fiches animales, le cycle de vie des rendez-vous et la surveillance des accès et de la sécurité.

À travers la réalisation de ce projet, j’ai pu mobiliser et consolider mes connaissances en **développement web** (maîtrise du langage Python, interaction avec les bases de données, implémentation des mécanismes de sécurité) et approfondir mes compétences en **analyse des besoins** et en **conception d’interfaces utilisateur**.

Ce travail démontre qu’une solution numérique simple et bien conçue peut significativement **améliorer l’organisation interne** d’une structure professionnelle et, par conséquent, **optimiser la qualité des services** offerts aux clients.

Ce stage s’est révélé être une expérience particulièrement enrichissante, me permettant une montée en compétence significative tant sur le plan technique que professionnel.